Modelos mecanicistas y estadísticos para brotes epidémicos: El caso de la COVID-19 en Euskadi

Tutorial en R - parte I

Dae-Jin Lee
BCAM - Basque Center for Applied Mathematics
Seminario Internacional EUSTAT 2020

Vitoria-Gasteiz, Nov 2020

Preliminares

- Todo el material de este tutorial (transparencias y código R) está disponible a través del siguiente enlace.

https://idaejin.github.io/courses/eustat2020/material/index.html

- **Requisitos:**
  - Conocimientos básicos del software estadístico R.
  - Última versión del software R (https://www.r-project.org).
  - Rstudio (https://www.rstudio.com).
- **Librerías necesarias**

```
install.packages(c("deSolve","bbmle",
                   "tidyverse","ggplot2","dplyr",
                   "lubridate","shiny","shinySIR",
                   "outbreaks","coronavirus"))
```

Objetivo de este Tutorial

Estructura de la sesión

1. Algunos recursos web.
2. Librerías de ‘R‘ para este tutorial
3. Modelo SIR.
4. Número reproductivo básico e instantáneo.
5. Ejemplos.

Recursos web sobre COVID-19 en R

Enlace https://www.repidemicsconsortium.org/

R Epidemics Consortium

RECON

The **R** Epidemics **Con**sortium (RECON) is an international not-for-profit, **non-governmental organisation** gathering experts in data science, modelling methodology, public health, and software development to create the next generation of analytics tools for informing the response to *disease outbreaks*, *health emergencies* and *humanitarian crises*, using the R software and other free, open-source resources.

Released projects and packages

These projects are in a usable form. Packages have been developed following RECON's standards, are fully functional, documented and tested, and have been released on CRAN.

distcrete
Discretized probability distributions.

EpiEstim
Quantifying transmissibility throughout an epidemic from incidence time series.

earlyR
Estimation of infectiousness in the early stage of an outbreak

epicontacts
Handling, visualisation and analysis of epidemiological contacts.

epitrix
Small helpers and tricks for epidemics analysis

incidence
Computation, handling, visualisation and simple modelling of incidence.

aweek
Convert dates to arbitrary week definitions including epiweek, isoweek, and more

outbreaker2
Modular framework for outbreak reconstruction.

outbreaks
Collection of outbreak data.

projections
Projections of future incidence

RECON learn
Open training platform for epidemics analysis

epiflows
Visualisation and analysis of passenger flows

Recursos web sobre COVID-19 en R

Enlace https://rviews.rstudio.com/tags/covid-19/

R Views Home About Contributors

An R community blog edited by R Studio

Boston, MA

298 POSTS | 262 TAGS

COVID-19

Help Delphi's COVIDcast Project fight the pandemic!
Joseph Rickert 2020-10-13

Monitor COVID-19 at the COVID-19 Forecast Hub
Joseph Rickert 2020-08-10

Some Upcoming R Related, Virtual Events
Joseph Rickert 2020-05-08

Some Select COVID-19 Modeling Resources
Joseph Rickert 2020-04-07

Simulating COVID-19 interventions with R
Tim Churches 2020-03-19

COVID-19 epidemiology with R
Tim Churches 2020-03-05

Librerías de R para este tutorial

deSolve: Solvers for Initial Value Problems of Differential Equations ('ODE', 'DAE', 'DDE')

```
library("deSolve")
?ode
```

- La función `ode()` (para ecuaciones diferenciales ordinarias).
- El paquete `deSolve` facilita la resolución del sistema de ecuaciones, y para encontrar los valores óptimos de los parámetros que deseamos estimar, podemos utilizar la función `optim()` incorporada de base en R.

Librerías de R para este tutorial (Cont.)

bbmle: Tools for General Maximum Likelihood Estimation

```
library("bbmle")
?mle2
```

- La función `mle2()` (permite estimar parámetros por el método de máxima verosimilitud).
- Usaremos también las funciones:
  - `coef()` (para extraer los coeficientes estimados),
  - `confint()` (para obtener intervalos de confianza),
  - `vcov()` (para extraer matrices de varianzas-covarianzas de los parámetros), o
  - `logLik()` (para extraer la log-verosimilud de un modelo ajustado)
  - `AIC`, `BIC`, etc.

Librerías de R para este tutorial (Cont.)

`outbreaks`: A Collection of Disease Outbreak Data

```
library("outbreaks")
```

- Datos reales o simulados de brotes epidémicos.

Librerías de R para este tutorial (Cont.)

`coronavirus`: The 2019 Novel Coronavirus COVID-19 (2019-nCoV) Dataset

```
library("coronavirus")
?coronavirus
```

- Proporciona un conjunto de datos en un formato `tidy` de la epidemia de COVID-19 (2019-nCoV).
- Permite extraer los datos la *Johns Hopkins University Center for Systems Science and Engineering (JHU CCSE)* https://coronavirus.jhu.edu/
- El paquete recoge variables como los casos confirmados, la muerte y los recuperados en diferentes países y estados.

Librerías de R para este tutorial (Cont.)

shinySIR: Interactive Plotting for Mathematical Models of Infectious Disease Spread

```
library("shinySIR")
?run_shiny()
```

- Proporciona una interfaz interactiva (basada en `shiny`) para modelos matemáticos de la propagación de enfermedades infecciosas.
- Los usuarios pueden elegir entre una variedad de modelos comunes de ecuaciones diferenciales ordinarias incorporadas (ODE) (entre ellos el modelo SIR), o crear los suyos propios.
- El paquete es una herramienta de enseñanza útil, ya que permite visualizar cómo el cambio de diferentes parámetros puede impactar en la dinámica del modelo.

El Modelo SIR

- **Compartimentos** (Nº de individuos) **S**usceptibles, **I**nfectados y **R**ecuperados.
- **Población total**: $N = S + I + R$
- **Parámetros**:
  - $\beta =$ "tasa de contagio" (*infectious contact rate*).
  - $\gamma =$ "tasa de recuperación" (*recovery rate*).

Veamos matemáticamente cómo representar este modelo SIR.

Descripción matemática del modelo SIR

Formalmente, se representa como un conjunto de ecuaciones diferenciales ordinarias (del inglés ODE - *Ordinary Differential Equations*):

$$\begin{aligned}
\frac{dS}{dt} &= -\beta S(t)\frac{I(t)}{N} \\
\frac{dI}{dt} &= \beta S(t)\frac{I(t)}{N} - \gamma I(t) \\
\frac{dR}{dt} &= \gamma I(t)
\end{aligned}$$

La dinámica del compartimento de infecciosos depende de la siguiente proporción (número reproductivo básico, R_0):

$$R_0 = \frac{\beta}{\gamma}$$

Modelo SIR en R

Resolver las ecuaciones diferenciales ordinarias

- Encontrar los valores de las variables (S, I y R) en varios momentos del tiempo t.
- Estos valores dependerán de los valores de los parámetros β y γ.
- Lo haremos de manera numérica a través de la función `ode()`.

```
library("deSolve")
?ode
```

Modelo SIR en R

Paso 1: escribir la ODE en R

```
sir_equations <- function(time, variables, parameters){
  with(as.list(c(variables, parameters)), {
    dS <- -beta * S * I/N
    dI <-  beta * S * I/N - gamma * I
    dR <-  gamma * I
    return(list(c(dS, dI, dR)))})
}
```

Modelo SIR en R

Paso 1: escribir la ODE en R

```
sir_equations <- function(time, variables, parameters){
  with(as.list(c(variables, parameters)), {
    dS <- -beta * S * I/N
    dI <-  beta * S * I/N - gamma * I
    dR <-  gamma * I
    return(list(c(dS, dI, dR)))})
}
```

- Esta función (ha de pasar como input en `ode()`) se compone de:
  - `time` es un vector que contiene la secuencia de momentos en el tiempo (e.g. días) sobre los que evaluamos el modelo SIR.
  - `variables` es un vector que contiene las variables S, Iy R en cada momento t.
  - `parameters` es un vector que contiene el valor de β y γ.

Modelo SIR en R

Paso 2: Definir unos valores para los parámetros

```
N <- 1000
parameters_values <- c(
   beta = 0.4,
  gamma = .05
)
```

Modelo SIR en R

Paso 2: Definir unos valores para los parámetros

```
N <- 1000
parameters_values <- c(
   beta = 0.4,
  gamma = .05
)
```

Paso 3: Definir unos valores iniciales para las variables

```
initial_values <- c(
  S = N-1,  # Nº de susceptibles en t = 0
  I =   1,  # Nº de infecciosos en t = 0
  R =   0   # Nº de recuperados (en inmunes) en t = 0
)
```

Paso 4: Crear una secuencia de puntos

```
time_values <- 1:100 # Días
```

```
parameters_values
##  beta gamma
##  0.40  0.05
initial_values
##   S   I   R
## 999   1   0
time_values
##   [1]   1   2   3   4   5   6   7   8   9  10  11  12  13  14  15  16  17  18
##  [19]  19  20  21  22  23  24  25  26  27  28  29  30  31  32  33  34  35  36
##  [37]  37  38  39  40  41  42  43  44  45  46  47  48  49  50  51  52  53  54
##  [55]  55  56  57  58  59  60  61  62  63  64  65  66  67  68  69  70  71  72
##  [73]  73  74  75  76  77  78  79  80  81  82  83  84  85  86  87  88  89  90
##  [91]  91  92  93  94  95  96  97  98  99 100
```

Modelo SIR en R

Paso 5: Resolver la ecuación con `ode()`

```
sir_values_1 <- ode(
      y = initial_values,
  times = time_values,
   func = sir_equations,
  parms = parameters_values
)
```

Modelo SIR en R

Paso 5: Resolver la ecuación con `ode()`

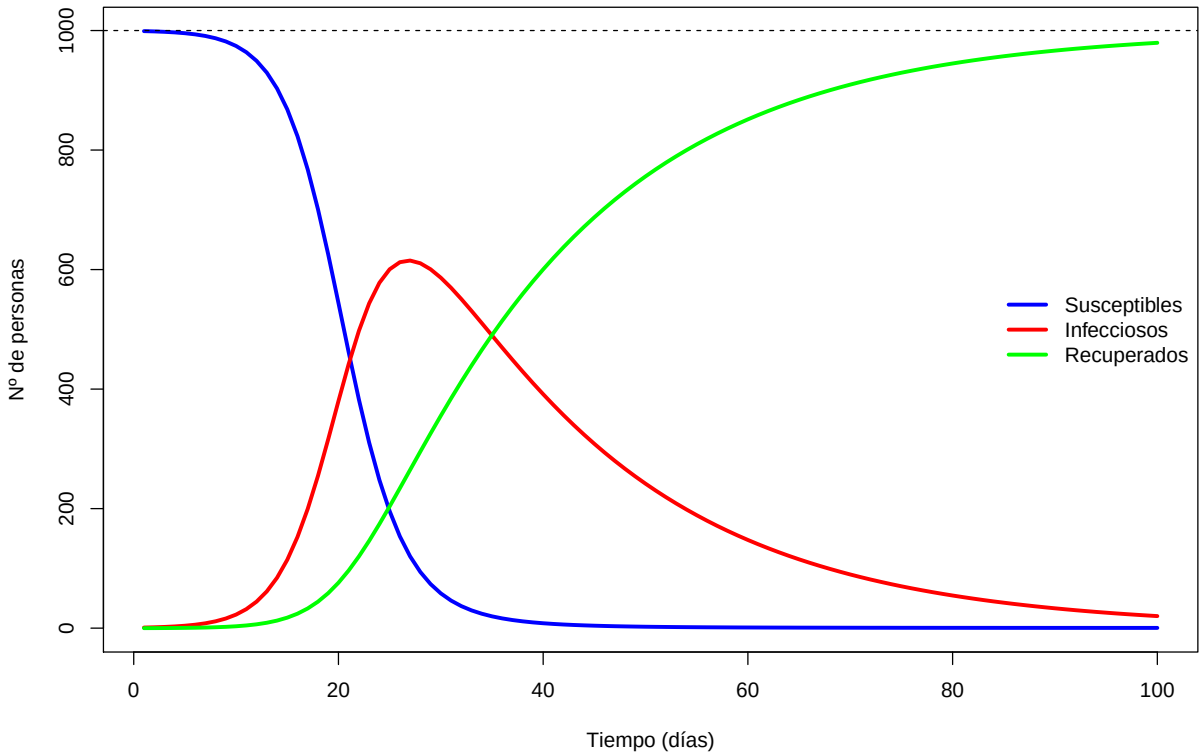
```
sir_values_1 <- ode(
      y = initial_values,
  times = time_values,
   func = sir_equations,
  parms = parameters_values
)
```

```
head(sir_values_1)
##      time        S        I          R
## [1,]    1 999.0000 1.000000 0.00000000
## [2,]    2 998.5218 1.418375 0.05985259
## [3,]    3 997.8439 2.011323 0.14473574
## [4,]    4 996.8837 2.851228 0.26508486
## [5,]    5 995.5243 4.040011 0.43565090
## [6,]    6 993.6020 5.720722 0.67725384
```

Modelo SIR en R

Veamos el resultado gráficamente

```
matplot(sir_values_1[,1],sir_values_1[,2:4], col = c("blue","red","green"),
        type="l",lty=1,lwd=3, xlab = "Tiempo (días)", ylab = "Nº de personas")
legend("right", c("Susceptibles", "Infecciosos", "Recuperados"),
       col = c("blue", "red", "green"), lty = 1, lwd = 3, bty = "n")
abline(h = N, lty = 2)
```



Modelo SIR en R

El valor del R_0 sería $R_0 = \frac{\beta}{\gamma}$, en este ejemplo $R_0 = 0.4/0.05 = 8$

```
R0 <- setNames(parameters_values["beta"] / parameters_values["gamma"],"R0")
R0
## R0
##  8
```

Creando una función propia `SIR` en `R`

Vamos a escribir una función propia que incluya todos los pasos anteriores:

- **Inputs:**
  - Valores de los paramétros (β y γ).
  - Valores iniciales de las variables (S_0, I_0, R_0).
  - Vector de momentos en el tiempo (t)
- **Output:**
  - Un `data.frame` con la solución de la ODE.

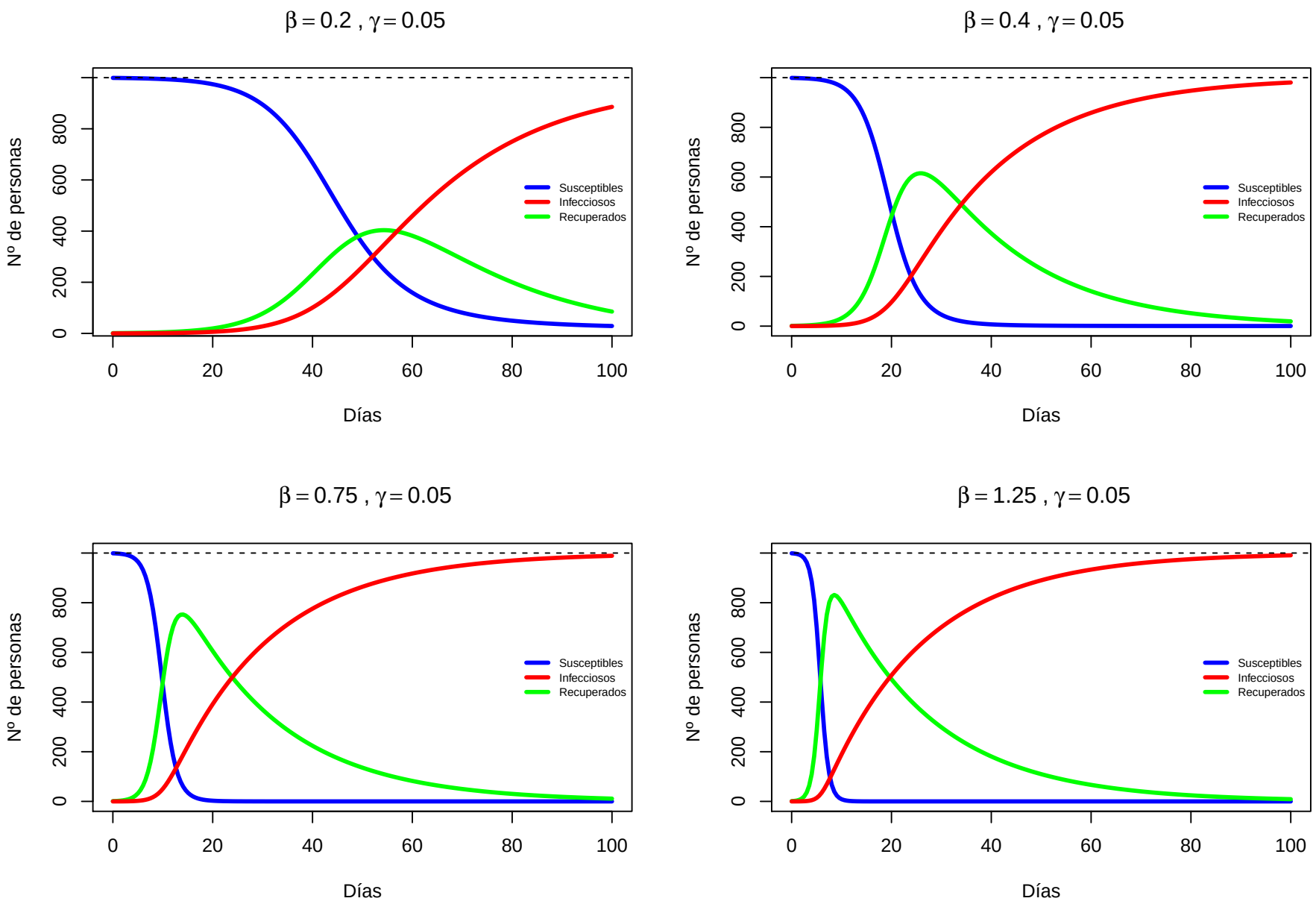
Creando una función propia SIR en R

```
mySIR <- function(beta, gamma, S0, I0, R0, times) {

  # the differential equations:
  sir_equations <- function(time, variables, parameters){
    with(as.list(c(variables, parameters)), {
      dS <- -beta * S * I / N
      dI <- beta * S * I / N - gamma * I
      dR <- gamma * I
      return(list(c(dS, dI, dR))) })
  }

  # the parameters values:
  parameters_values <- c(beta  = beta, gamma = gamma)

  # the initial values of variables:
  initial_values <- c(S = S0, I = I0, R = R0)

  # solving
  out <- ode(initial_values, times, sir_equations, parameters_values)

  # returning the output:
  as.data.frame(out)
}
```

Creando una función propia SIR en R

```
mySIR(beta = 0.4, gamma=0.05, S0=N-1, I0=1, R0=0, times = seq(0,100))
```

Con esta función `mySIR()` podemos variar los parámetros y analizar la dinámica de la epidemia.


β = 0.2 , γ = 0.05
β = 0.4 , γ = 0.05
β = 0.75 , γ = 0.05
β = 1.25 , γ = 0.05
Nº de personas
Días
Susceptibles
Infecciosos
Recuperados
0
200
400
600
800
0
20
40
60
80
100

Librería shinySIR

Con esta librería se pueden elegir distintos modelos de compartimentos para la propagación de enfermedades infecciosas basados en ODE's.

```
install.packages("shinySIR")
```

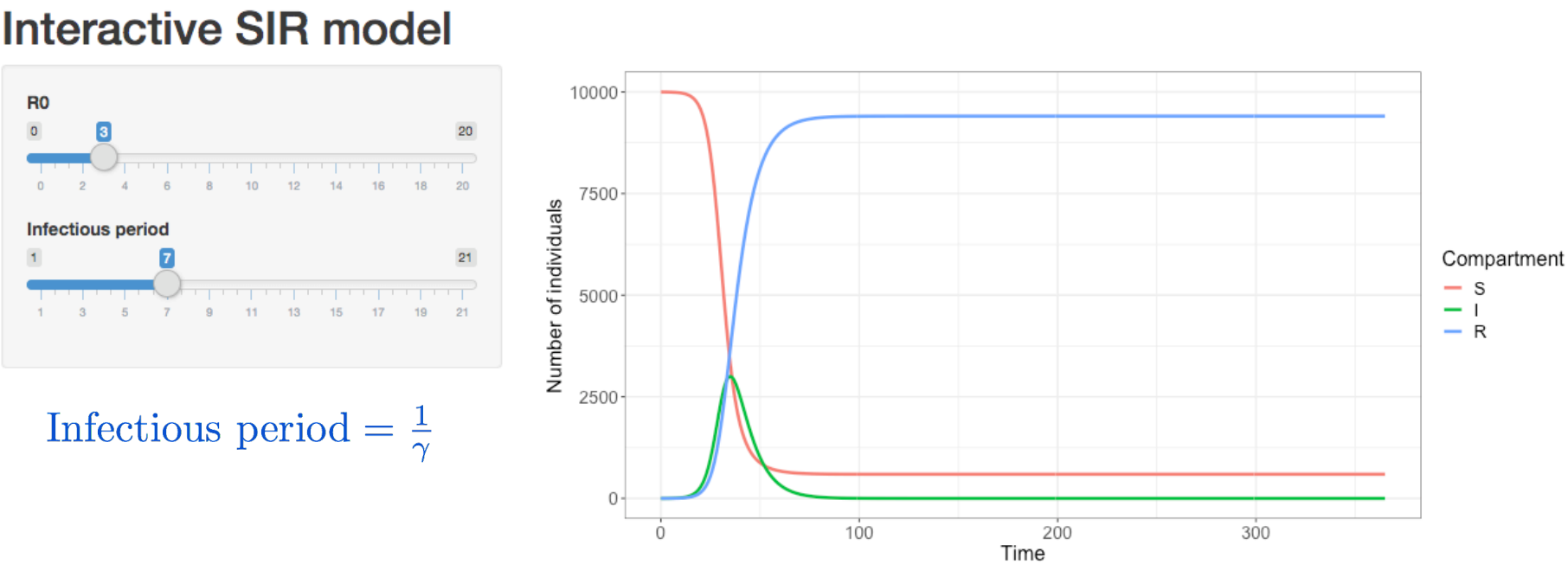
Por ejemplo, para el modelo SIR

```
library(shinySIR)
run_shiny(model = "SIR")
```

Más modelos de ODE's en https://cran.r-project.org/web/packages/shinySIR/vignettes/Vignette.html

También es posible especificar otros modelos siguiendo la sintaxis de la función `ode()`.

Librería shinySIR



$$\text{Infectious period} = \frac{1}{\gamma}$$

Parameter	Value
gamma	1.4E-01
beta	4.3E-05

$$\beta = R_0 \gamma / N$$

Modelo SIR y ajuste de datos

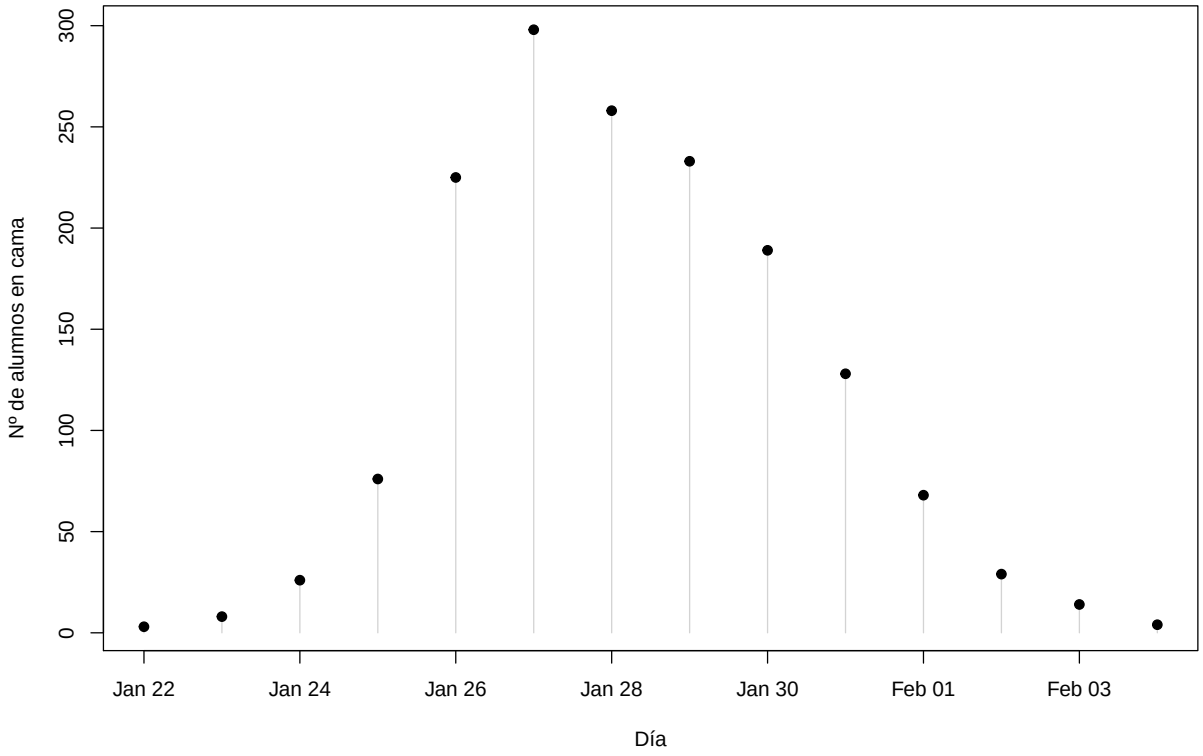
- Vamos a ajustar el modelo SIR a unos datos de gripe en un colegio en Inglaterra en 1978.

```
library(outbreaks)
data(influenza_england_1978_school)
head(influenza_england_1978_school)
##         date in_bed convalescent
## 1 1978-01-22      3            0
## 2 1978-01-23      8            0
## 3 1978-01-24     26            0
## 4 1978-01-25     76            0
## 5 1978-01-26    225            9
## 6 1978-01-27    298           17
```

- **Periodo:** entre 1978-01-22 y 1978-02-04 (14 días).
- **Infectados:** un total de 512 alumnos de 763 enfermaron.

Modelo SIR y ajuste de datos

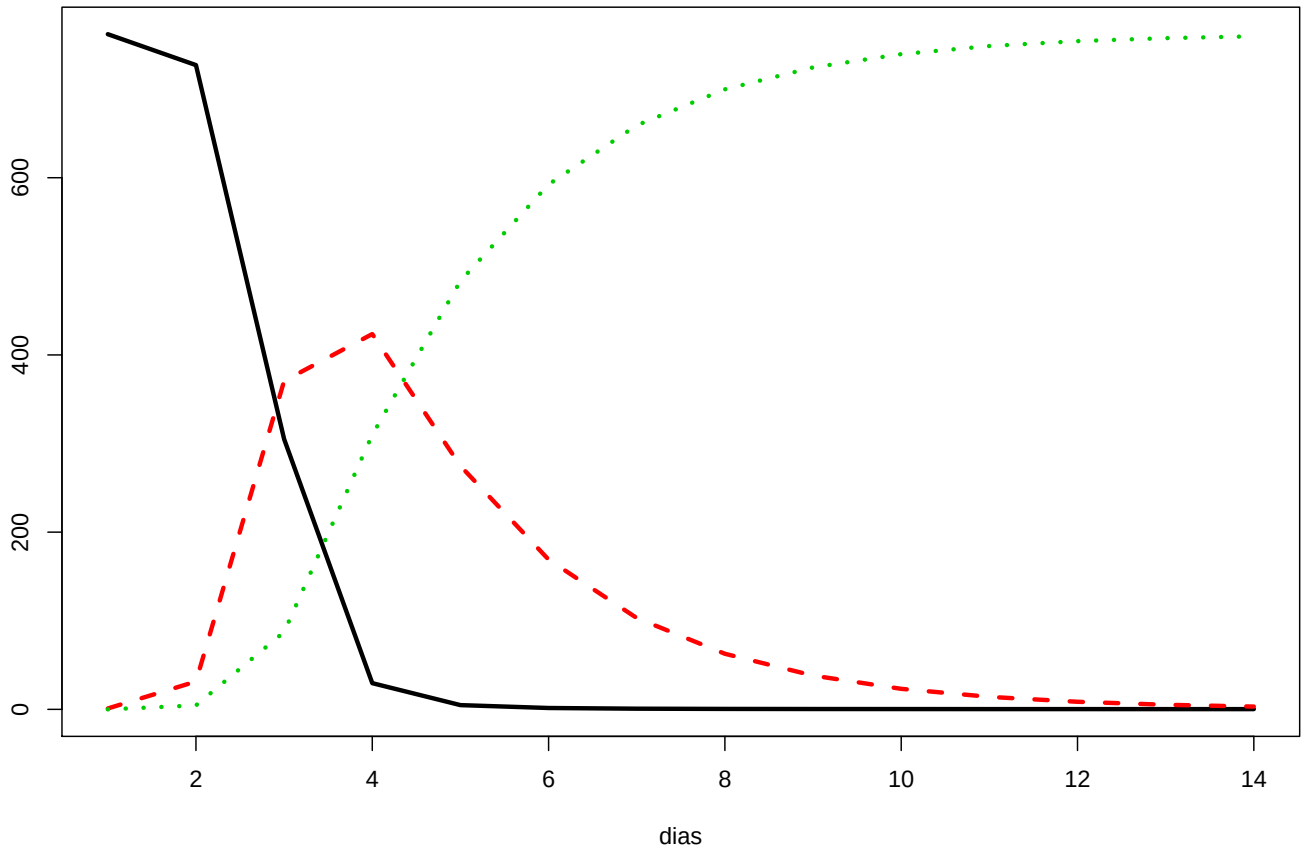
```
datos <- influenza_england_1978_school
plot(datos[,c(1,2)], xlab = "Día",
     ylab = "Nº de alumnos en cama", type = "h", col = "lightgrey")
points(datos[,c(1,2)],pch=19)
```



Modelo SIR y ajuste de datos

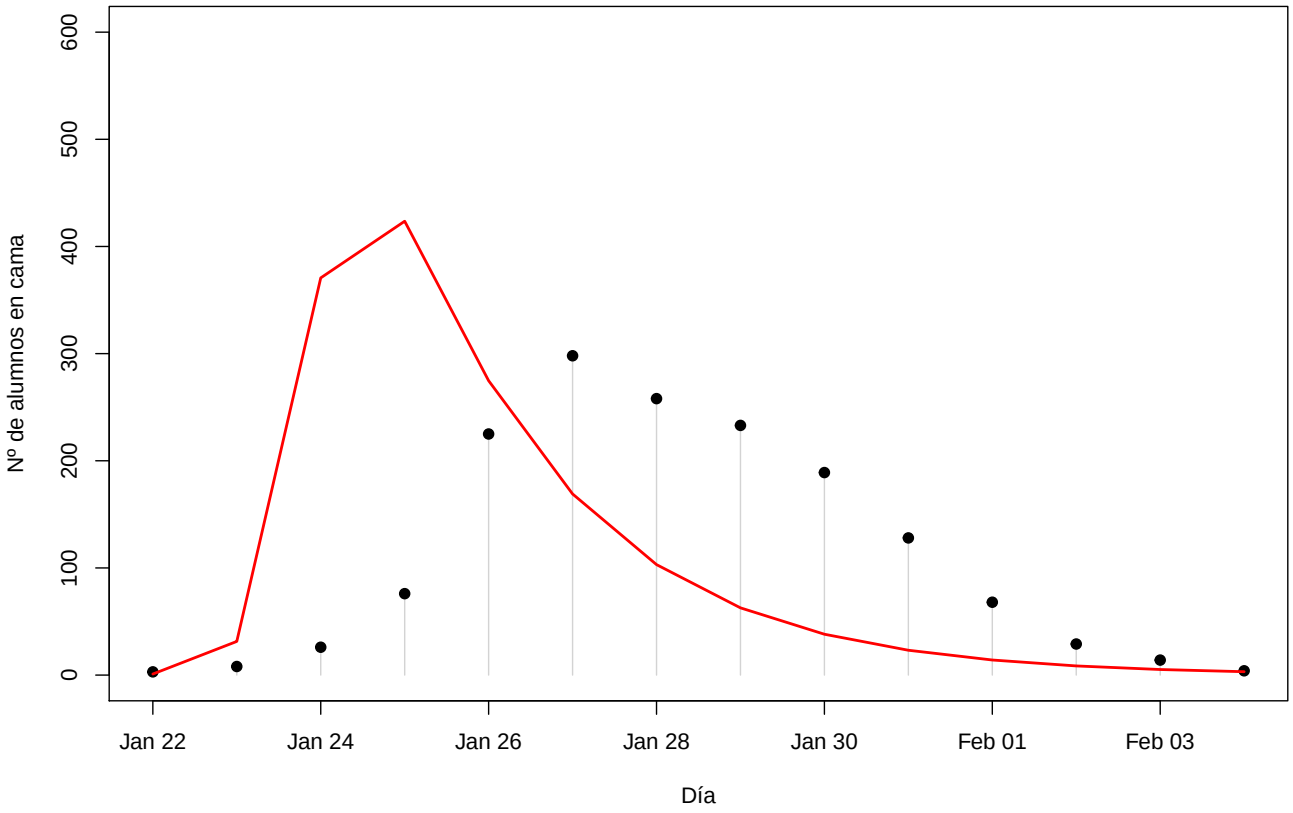
```
 dias <- seq(1,14)
casos <- datos$in_bed

N <- 763
predictions <- mySIR(beta = 4, gamma = 0.5,
                     S0 = N-1, I0 = 1, R0 = 0, times = dias)

head(predictions)
##   time          S        I          R
## 1    1 762.000000   1.0000   0.000000
## 2    2 727.121843  31.4096   4.468562
## 3    3 304.998002 370.6727  87.329338
## 4    4  29.581626 423.5652 309.853221
## 5    5   4.790725 274.7287 483.480615
## 6    6   1.528461 169.0326 592.438946
```

```
matplot(dias,predictions[,2:4],t="l",lwd=3, ylab ="")
```



Modelo SIR y ajuste de datos

```
plot(datos[,c(1,2)], xlab = "Día", ylab = "Nº de alumnos en cama", type = "h",
     col = "lightgrey",ylim = c(0,600))
points(datos[,c(1,2)],pch=19)
lines(datos$date,predictions$I, col = "red", lwd=2)
```

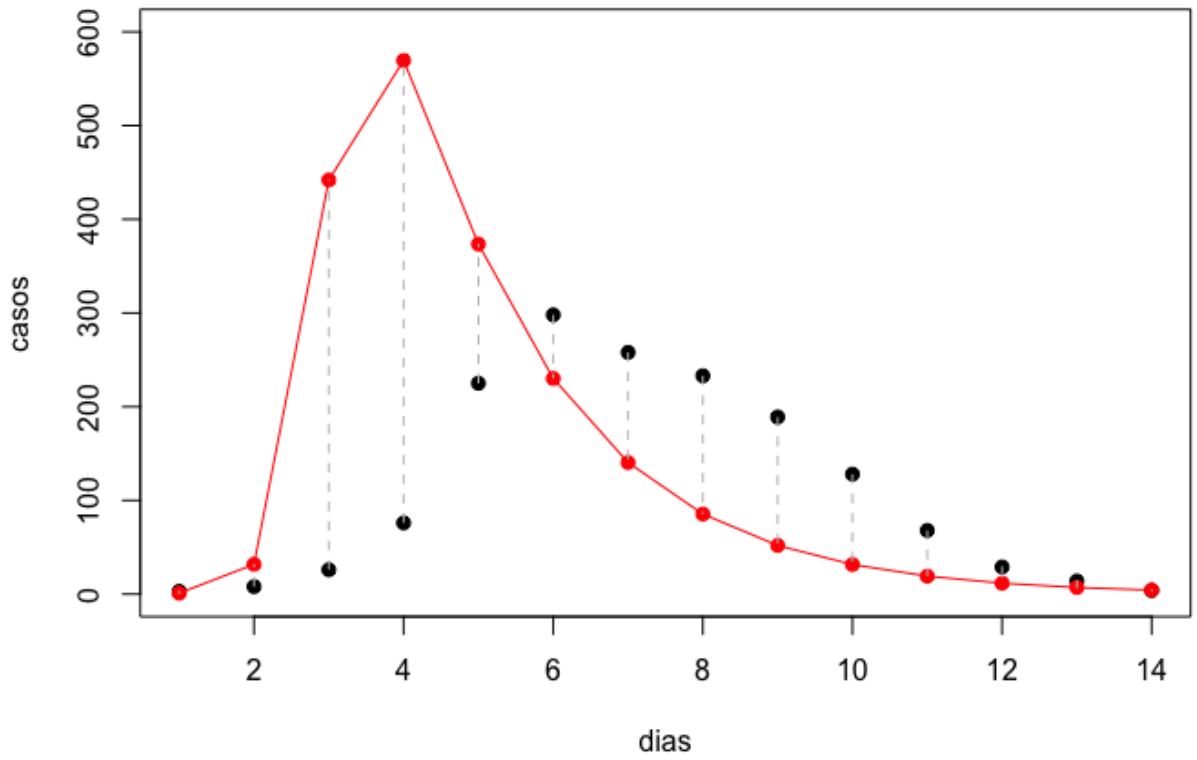


¿Cómo seleccionar β y γ para describir mejor la epidemia?

Estimar de manera óptima los parámetros del modelo SIR

- Una manera es minimizar la **Suma de Cuadrado de los Residuos** (SCR), entre los casos infectados (I) e infectados predichos ($\hat{I}$):

$$\text{SCR} = \sum^{n} (I - \hat{I})^2$$



Modelo SIR: Suma de Cuadrados de lor Residuos

- La función `SCR` es una función que calcula la suma de cuadrados de los residuos de un modelo SIR para los datos de gripe.
- Incluye la función `mySIR`.

```
SCR <- function(beta, gamma, data = datos, N = 763) {
  I0 <- datos$casos[1] # Valores iniciales de I
  times <- datos$dias
  predictions <- mySIR(beta = beta, gamma = gamma,   # parameters
                       S0 = N - I0, I0 = I0, R0 = 0, # variables' intial values
                       times = times)                # time points
  out <- sum((predictions$I[-1] - datos$casos[-1])^2)
  return(out)
}
```

Modelo SIR: Suma de Cuadrados de lor Residuos

```
datos <- influenza_england_1978_school
datos$dias <- seq(1,14) # vector de días (numérico)
colnames(datos)[2] <- "casos" # renombro la columna in_bed -> casos

head(datos)
##         date casos convalescent dias
## 1 1978-01-22     3            0    1
## 2 1978-01-23     8            0    2
## 3 1978-01-24    26            0    3
## 4 1978-01-25    76            0    4
## 5 1978-01-26   225            9    5
## 6 1978-01-27   298           17    6
```

```
res <- SCR(beta = 4, gamma = 0.5, data = datos, N = 763)
res
## [1] 406083.9
```

Modelo SIR: búsqueda de valores óptimos de β y γ

- Vamos a "buscar" los valores de β y γ que minimicen la SCR.

Modelo SIR: búsqueda de valores óptimos de β y γ

- Vamos a "buscar" los valores de β y γ que minimicen la SCR.
- Por ejemplo, en los intervalos: $\beta \in [1, 3]$ y $\gamma \in [0.4, 0.5]$.

```
 beta_val <- seq(from = 1, to = 3, l = 100)
gamma_val <- seq(from = 0.4, to = 0.5, l = 100)
```

Modelo SIR: búsqueda de valores óptimos de β y γ

- Vamos a "buscar" los valores de β y γ que minimicen la SCR.
- Por ejemplo, en los intervalos: $\beta \in [1, 3]$ y $\gamma \in [0.4, 0.5]$.

```
 beta_val <- seq(from = 1, to = 3, l = 100)
gamma_val <- seq(from = 0.4, to = 0.5, l = 100)
```

- Vamos a evaluar valores de β para un valor de $\gamma = 0.5$

```
ss_val_beta <- sapply(beta_val, SCR, gamma = 0.5)
min(ss_val_beta) # Valor mínimo de la SCR
## [1] 7326.99
```

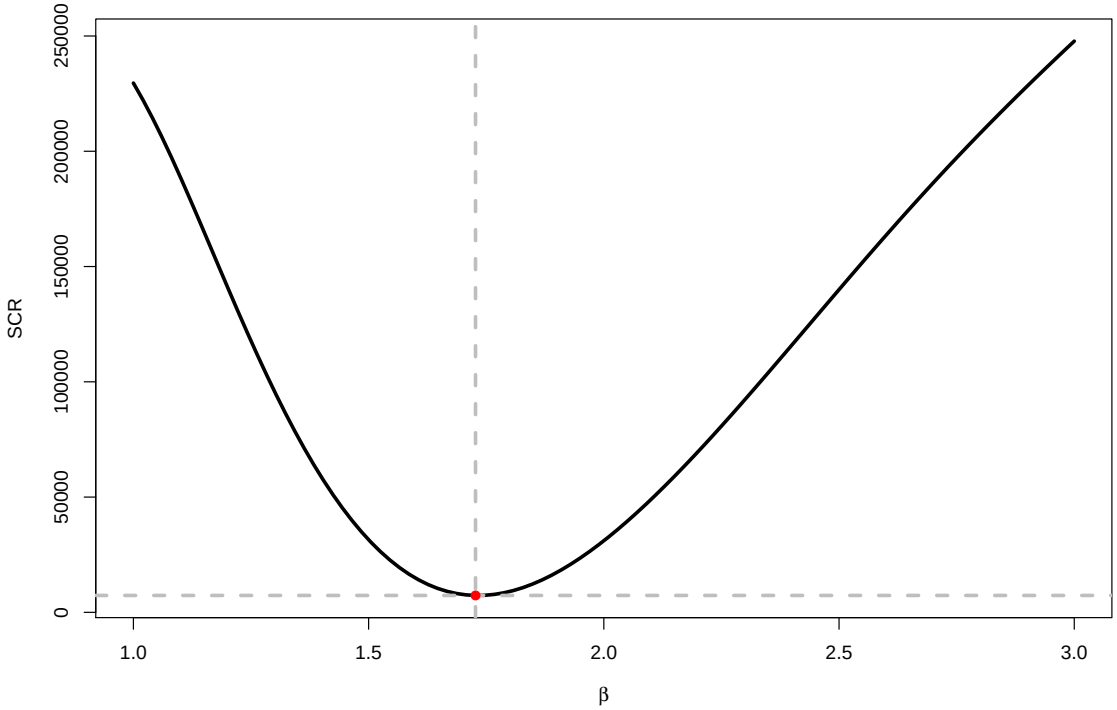
- El valor de $\hat{\beta}$ que minimiza la SCR (con $\gamma = 0.5$) es:

```
beta_hat <- beta_val[which.min(ss_val_beta)]
beta_hat
## [1] 1.727273
```

Modelo SIR: búsqueda de valores óptimos de β y γ

- Gráficamente:

```
plot(beta_val, ss_val_beta,
     xlab = expression(paste(beta)),
     ylab = "SCR", lwd=3, type="l")
abline(v = beta_hat,h = min(ss_val_beta), lty=2, lwd = 3, col = "grey")
points(beta_hat,min(ss_val_beta),col="red",pch=19)
```



Modelo SIR: búsqueda de valores óptimos de β y γ

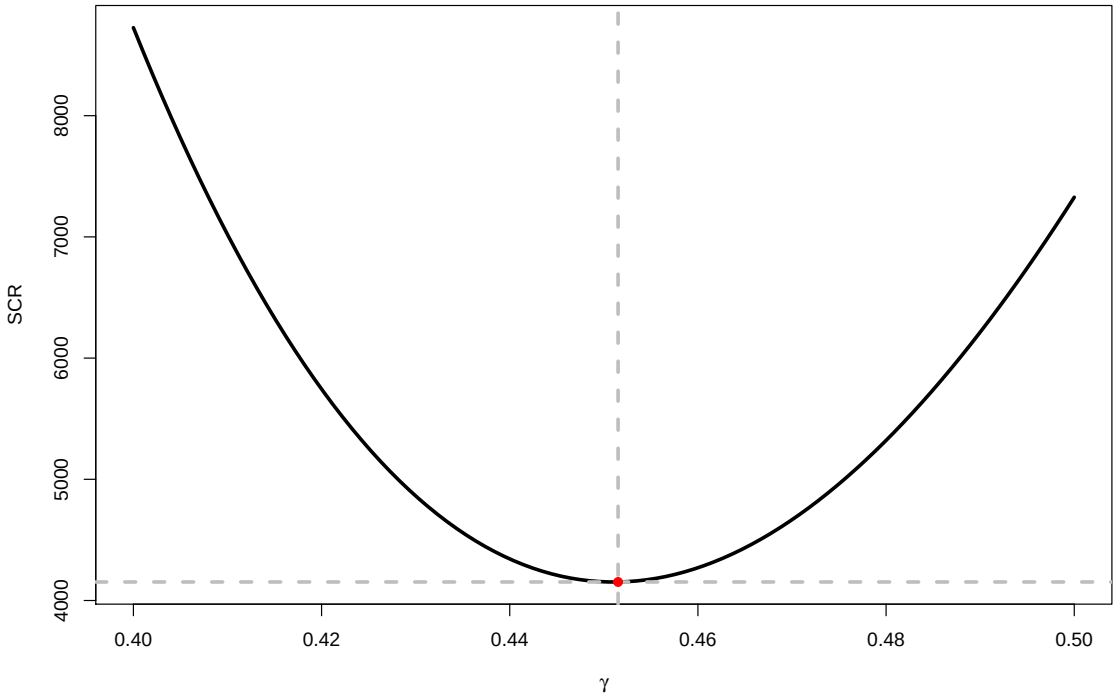
- Vamos a repetir el proceso para γ (con $\beta = 1.727$)

```
ss_val_gamma <- sapply(gamma_val, function(x) SCR(beta_hat, x))
min(ss_val_gamma)
## [1] 4152.821
gamma_hat <- gamma_val[which.min(ss_val_gamma)]
gamma_hat
## [1] 0.4515152
```

Modelo SIR: búsqueda de valores óptimos de β y γ

- Gráficamente

```
plot(gamma_val, ss_val_gamma,
     xlab = expression(paste(gamma)),
     ylab = "SCR", lwd=3, type="l")
abline(v = gamma_hat,h = min(ss_val_gamma), lty=2, lwd=3, col = "grey")
points(gamma_hat,min(ss_val_gamma),col="red",pch=19)
```

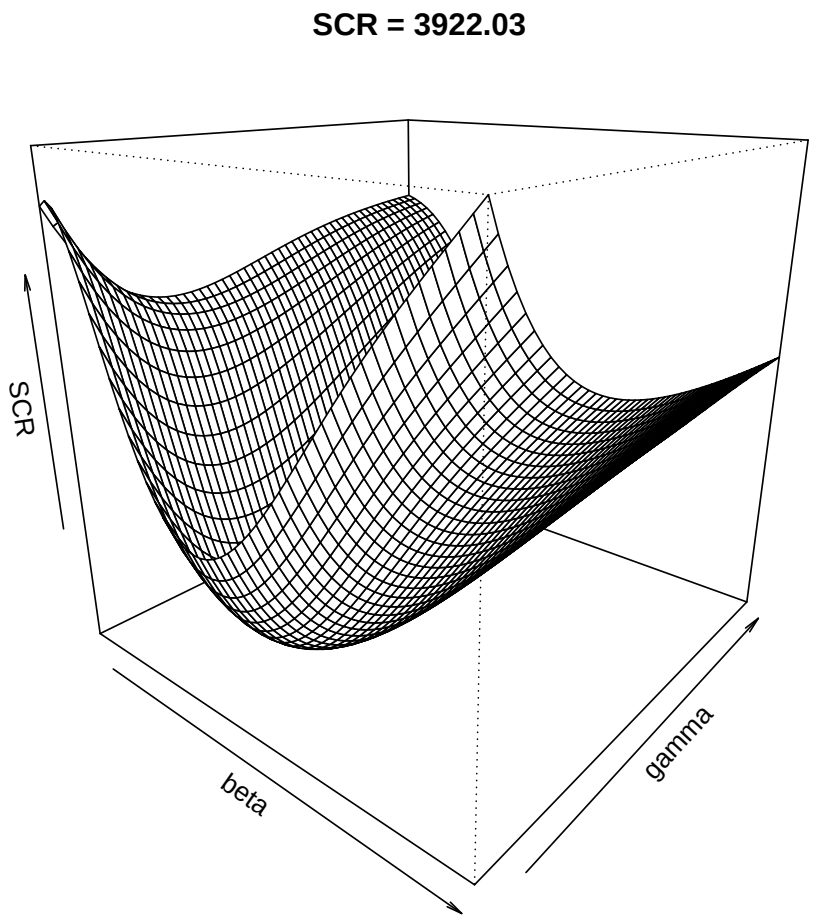


Modelo SIR: búsqueda de valores óptimos de β y γ

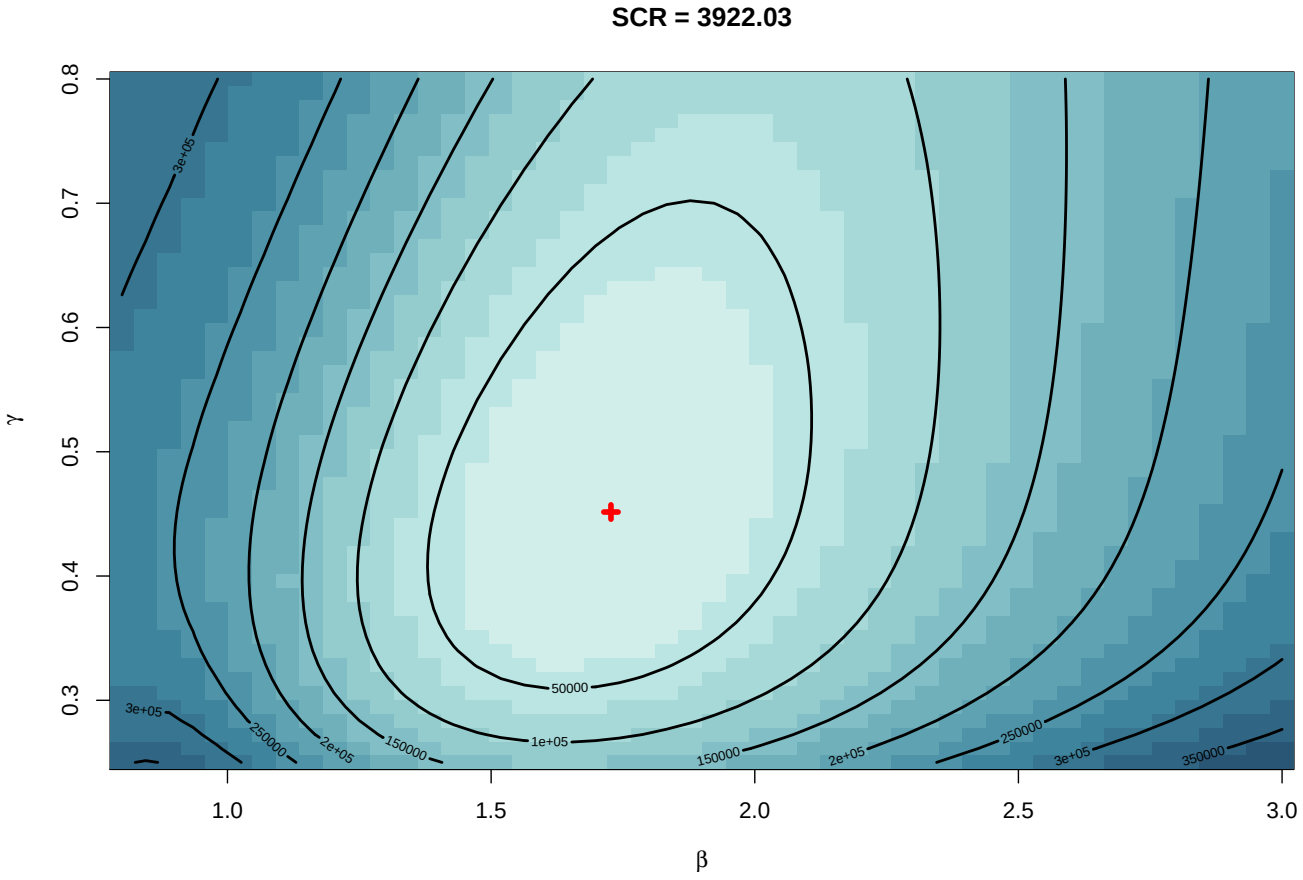
- ¿Cómo buscar simultáneamente β y γ?

```
        n <- 50
 beta_val <- seq(from = 0.8, to = 3, l = n)
gamma_val <- seq(from = 0.25, to = 0.8, l = n)
param_val <- expand.grid(beta=beta_val,gamma=gamma_val)
ss_val <- with(param_val, Map(SCR, beta, gamma))
ss_val <- matrix(unlist(ss_val), n)

min(ss_val)
## [1] 3922.029

param_val[which.min(ss_val),]
##         beta     gamma
## 921 1.697959 0.4520408
```

```
persp(beta_val, gamma_val, ss_val,
      theta = 40, phi = 20,
      xlab = expression(beta), ylab = expression(gamma),
      zlab = "SCR",
      main = paste0("SCR = ",round(min(ss_val),2)))
```



```
image(beta_val, gamma_val, ss_val, xlab = expression(beta),col= rev(hcl.colors(12,
      ylab = expression(gamma),main = paste0("SCR = ",round(min(ss_val),2)))
contour(beta_val, gamma_val, ss_val,add=TRUE,col="black",lwd=2,nlevels = 10)
points(beta_hat, gamma_hat, pch = 3, lwd = 4, col="red")
```



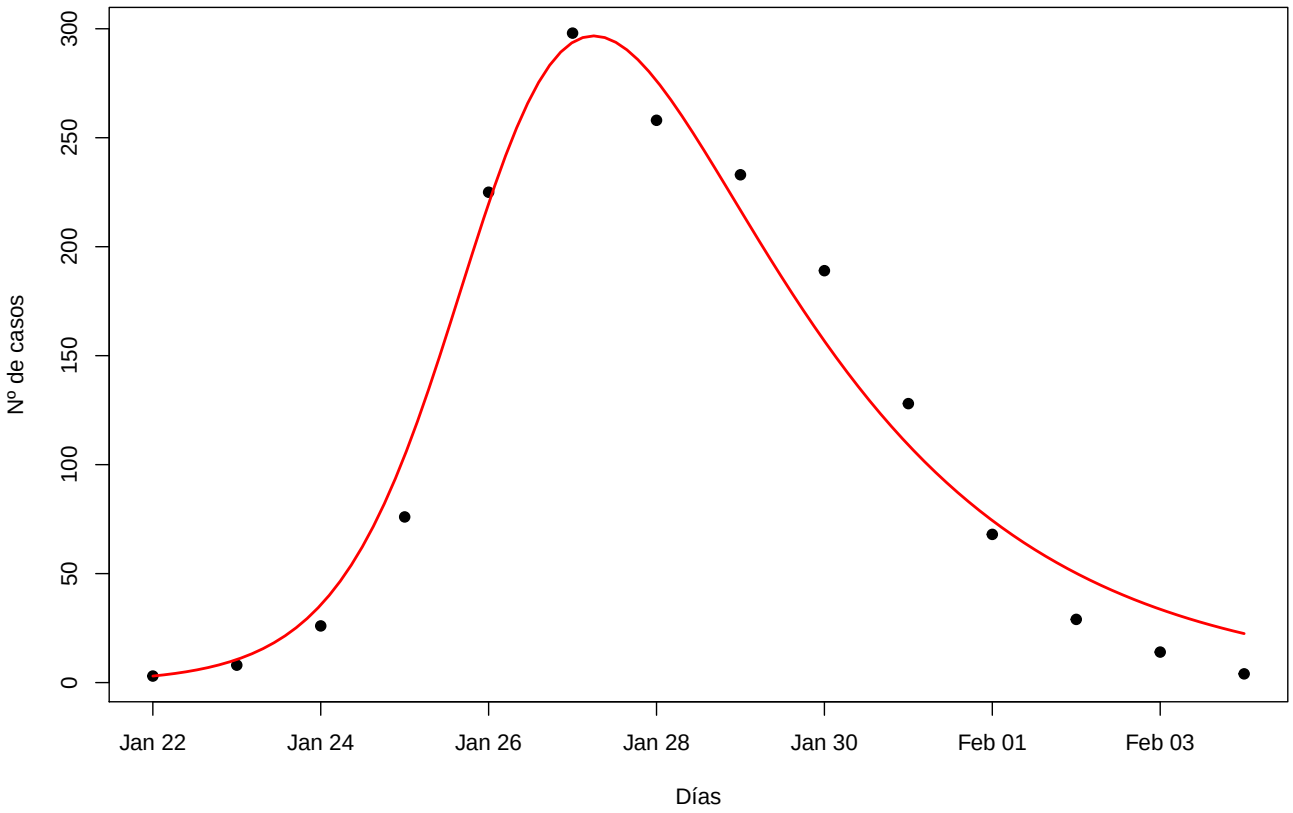
Modelo SIR

- Vamos a ver el ajuste sobre los datos de la gripe en el colegio inglés.

```
 N <- 763                # Población total
I0 <- datos$casos[1]  # Infecciosos (valor inicial)
S0 <-  N-I0           # Susceptibles (valor inicial)

time_points <- seq(1,14,l=100)  # 100 valores entre 1 y 14 días

# Ajuste del modelo SIR
fit <- mySIR(beta = beta_hat, gamma = gamma_hat,
             S0 = S0, I0 = I0, R0 = 0, times = time_points)
```

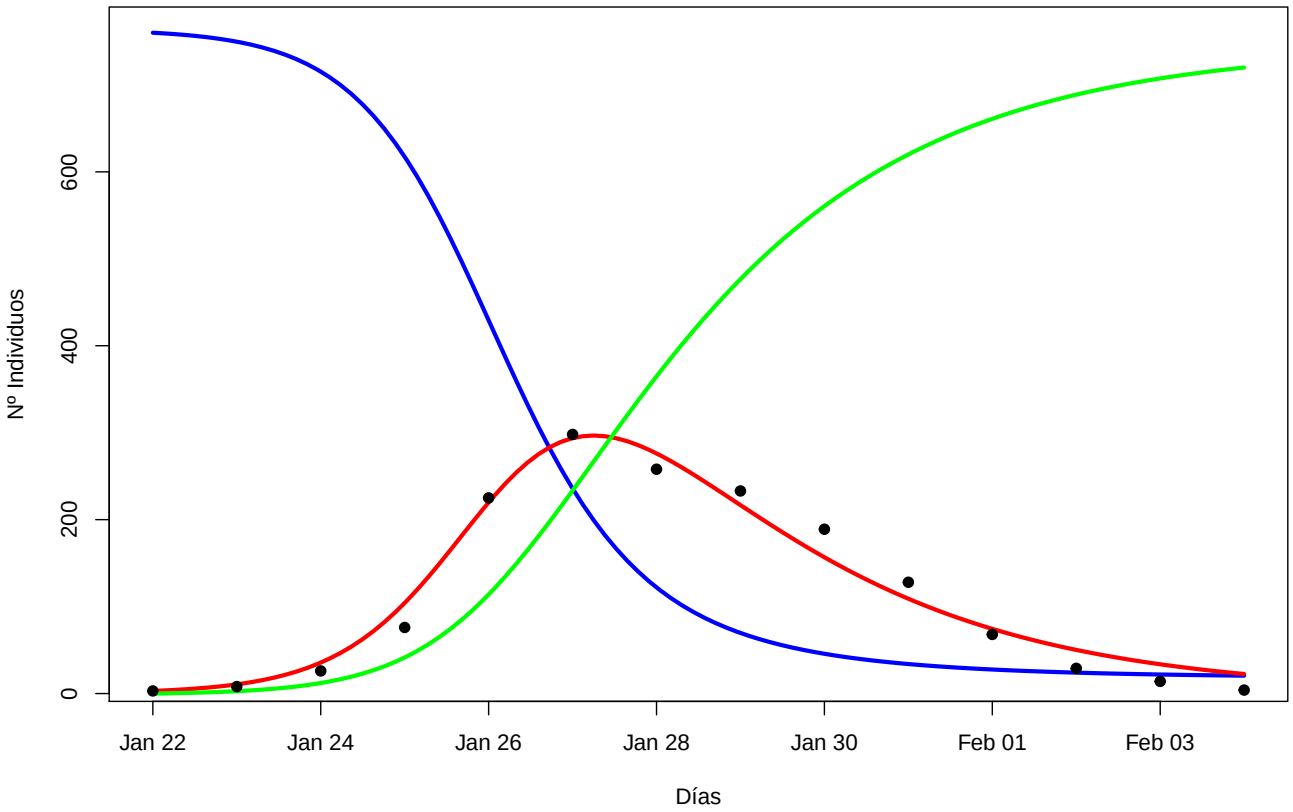
Modelo SIR

```
# vector de fechas (para el gráfico)
dates_points <- seq(min(datos$date),max(datos$date),l=100)

plot(datos$date,datos$casos,col="black",pch=19,
     xlab = "Días", ylab = "Nº de casos")
lines(dates_points,fit$I,col="red", lwd = 2)
```



Modelo SIR

```
plot(dates_points,fit$S,t="l",col=c("blue"),lwd=3,
     xlab = "Días", ylab = "Nº Individuos")
lines(dates_points,fit$I,col="red",lwd=3)
lines(dates_points,fit$R,col="green",lwd=3)
points(datos$date,datos$casos,pch=19,col="black")
```



Modelo SIR: optimización mediante la función `optim()`

- Vamos a estimar los parámetros de manera más eficiente.
- La función `optim()`, permite buscar los parámetros β y γ de manera "inteligente". Necesitamos:
  - Crear un vector de valores iniciales que queremos optimizar.
  - La función a minimizar (en este caso la SCR).

Modelo SIR: optimización mediante la función `optim()`

- Vamos a estimar los parámetros de manera más eficiente.
- La función `optim()`, permite buscar los parámetros β y γ de manera "inteligente". Necesitamos:
  - Crear un vector de valores iniciales que queremos optimizar.
  - La función a minimizar (en este caso la SCR).

- Necesitaremos modificar la función `SCR` y adaptarla para la función `optim()`.

```
SCR2<- function(params){
  out <- SCR(beta = params[1], gamma = params[2], data=datos, N=763)
  return(out)
}

params <- c(1, 0.5)
SCR2(params)
## [1] 229583
```

```r
init_params <- c(1,0.45)
ss_opt <- optim(init_params, SCR2)
ss_opt
## $par
## [1] 1.6997628 0.4468081
##
## $value
## [1] 3874.375
##
## $counts
## function gradient
##       59       NA
##
## $convergence
## [1] 0
##
## $message
## NULL
```

Modelo SIR

```
# Extraemos los parámetros de optim
 beta_hat2 <- ss_opt$par[1]
gamma_hat2 <- ss_opt$par[2]

# Ajuste del modelo SIR
fit2 <- mySIR(beta = beta_hat2, gamma = gamma_hat2,
             S0 = S0, I0 = I0, R0 = 0, times = time_points)

# Comparamos los ajustes con el método de búsqueda y con optim()
plot(dates_points,fit$I,t="l",col="red",lwd=3, xlab = "Días", ylab = "Nº Individuos
lines(dates_points,fit2$I,col="blue",lty=2,lwd=3)
points(datos$date,datos$casos,pch=19,col="black")
legend("topright", c("búsqueda parámetros", "optim()"),
       lwd=3, lty=1:3, col=c("red","blue"))
```

Modelos SIR: estimación por máxima verosimilitud

- Podemos utilizar métodos de máxima verosimilitud para estimar los parámetros de un modelo SIR.
- El **método de máxima verosimilitud (MV)** es una generalización de la idea de minimizar la suma de cuadrados de los residuos.

Modelos SIR: estimación por máxima verosimilitud

- Podemos utilizar métodos de máxima verosimilitud para estimar los parámetros de un modelo SIR.
- El **método de máxima verosimilitud (MV)** es una generalización de la idea de minimizar la suma de cuadrados de los residuos.

- Podemos definir una función de verosimilitud a maximizar en lugar de minimizar la SCR.
- Podemos escribir una función de verosimilitud en `optim()`.

Modelos SIR: estimación por máxima verosimilitud

- Podemos utilizar métodos de máxima verosimilitud para estimar los parámetros de un modelo SIR.
- El **método de máxima verosimilitud (MV)** es una generalización de la idea de minimizar la suma de cuadrados de los residuos.

- Podemos definir una función de verosimilitud a maximizar en lugar de minimizar la SCR.
- Podemos escribir una función de verosimilitud en `optim()`.

- En cambio, vamos a usar la librería `bblme` que se basa en la función `optim()` y permite fácilmente calcular intervalos de confianza, etc.

```
library(bbmle)
```

Modelos SIR: estimación por máxima verosimilitud

- Vamos a suponer que nuestros casos siguen una distribución normal.
- Ajustaremos una distribución Normal.

```
# Definimos una función de la log-verosimilud
mLL <- function(beta, gamma, sigma, dias, casos, N = 763) {
   beta <- exp(beta)    # para asegurar parámetros >0
  gamma <- exp(gamma)
  sigma <- exp(sigma)
  I0 <- casos[1]
  observations <- casos[-1] # ajustamos los datos restantes
  predictions <- mySIR(beta = beta, gamma = gamma,
                       S0 = N - I0, I0 = I0, R0 = 0, times = dias)
  predictions <- predictions$I[-1] # eliminamos el primer valor ajustado
# - log-verosimilitud de una Distribución Normal:
  -sum(dnorm(x = observations, mean = predictions, sd = sigma, log = TRUE))
}
```

Modelos SIR: estimación por máxima verosimilitud

- Necesitaremos definir unos valores iniciales.
- Usaremos la función `mle2`.

```
# Lista de valores iniciales de los parámetros
starting_param_val <- list(beta = 1, gamma = 0.30, sigma = 1)

# Estimamos con mle2
estimates <- mle2(minuslogl = mLL, start = lapply(starting_param_val, log),
                  method = "Nelder-Mead", data = c(datos, N = 763))
estimates
##
## Call:
## mle2(minuslogl = mLL, start = lapply(starting_param_val, log),
##     method = "Nelder-Mead", data = c(datos, N = 763))
##
## Coefficients:
##       beta      gamma      sigma
##  0.5308297 -0.8057162  2.8484051
##
## Log-likelihood: -55.48
```

Modelos SIR: estimación por máxima verosimilitud

- Necesitaremos definir unos valores iniciales.
- Usaremos la función `mle2`.

```
# Lista de valores iniciales de los parámetros
starting_param_val <- list(beta = 1, gamma = 0.30, sigma = 1)

# Estimamos con mle2
estimates <- mle2(minuslogl = mLL, start = lapply(starting_param_val, log),
                  method = "Nelder-Mead", data = c(datos, N = 763))
estimates
##
## Call:
## mle2(minuslogl = mLL, start = lapply(starting_param_val, log),
##     method = "Nelder-Mead", data = c(datos, N = 763))
##
## Coefficients:
##       beta      gamma      sigma
##  0.5308297 -0.8057162  2.8484051
##
## Log-likelihood: -55.48
```

```r
summary(estimates)
## Maximum likelihood estimation
##
## Call:
## mle2(minuslogl = mLL, start = lapply(starting_param_val, log),
##     method = "Nelder-Mead", data = c(datos, N = 763))
##
## Coefficients:
##        Estimate Std. Error z value     Pr(z)
## beta   0.530830   0.016573  32.029 < 2.2e-16 ***
## gamma -0.805716   0.032147 -25.064 < 2.2e-16 ***
## sigma  2.848405   0.196083  14.527 < 2.2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## -2 log L: 110.9564
```

- Los coeficientes se han calculado en la escala log
- Vamos a transformarlos en la escala original

```
exp(coef(estimates))
##       beta      gamma      sigma
##  1.7003425  0.4467678 17.2602308
```

- Los coeficientes se han calculado en la escala log
- Vamos a transformarlos en la escala original

```
exp(coef(estimates))
##       beta      gamma      sigma
##  1.7003425  0.4467678 17.2602308
```

- Y calcular sus intervalos de confianza al 95%

```
exp(confint(estimates))
##            2.5 %     97.5 %
## beta   1.6413089  1.7606428
## gamma  0.4179379  0.4788757
## sigma 12.2783425 26.8251801
```

- Matriz de varianzas-covarianzas de los coeficientes

```
vcov(estimates)
##                beta         gamma         sigma
## beta  2.746724e-04  1.566346e-04  2.453081e-05
## gamma 1.566346e-04  1.033418e-03 -1.701352e-05
## sigma 2.453081e-05 -1.701352e-05  3.844857e-02
```

- $-\log$-verosimilitud

```
-logLik(estimates)
## 'log Lik.' 55.4782 (df=3)
```

- Criterio de Akaike `AIC` ó `AICc` (corregido para muestras pequeñas)

```
AIC(estimates)
## [1] 116.9564
AICc(estimates)
## [1] 119.3564
```

- Criterio de información bayesiano `BIC`

```
BIC(estimates)
## [1] 118.8736
```

Modelo SIR: estimación por máxima verosimilitud

- A continuación vamos a ajustar el modelo SIR basado en la estimación MV

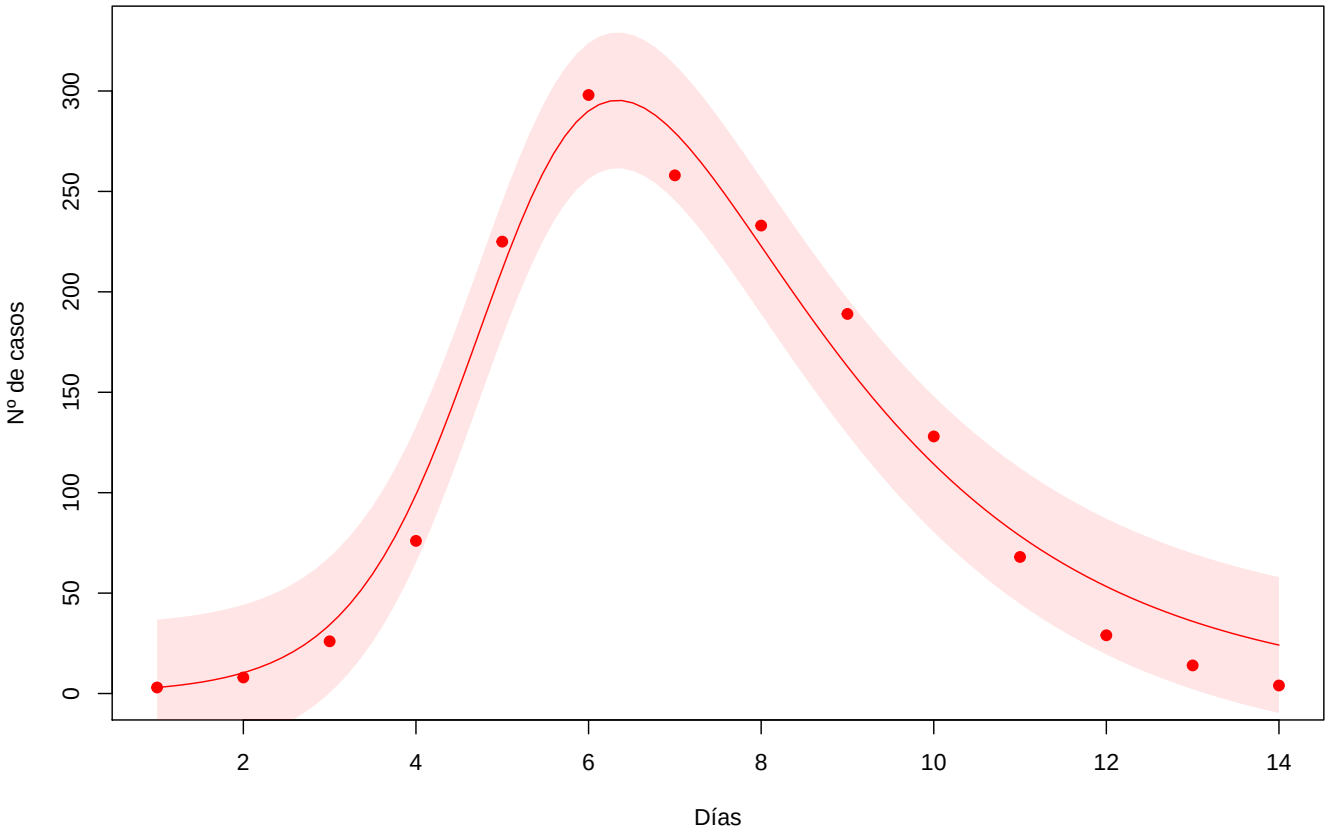
```
 beta_hat_mle <- exp(coef(estimates))[1]
gamma_hat_mle <- exp(coef(estimates))[2]
sigma_hat_lme <- exp(coef(estimates))[3]

# Ajustamos el modelo SIR con los parámetros obtenidos por MV
fit_mle <- mySIR(beta = beta_hat_mle,
                 gamma = gamma_hat_mle,
                 S0 = S0, I0 = I0, R0 = 0, times = time_points)
```

- Y calcular los intervalos de confianza del ajuste

```
low <- qnorm(p = 1-0.975,mean=fit_mle$I,sd = sigma_hat_lme)
 up <- qnorm(p = 0.975,mean=fit_mle$I,sd = sigma_hat_lme)
```

Modelo SIR: estimación por máxima verosimilitud

```
plot(datos$dias,datos$casos,col="red",pch=19, ylim = c(0,max(up)),
     xlab = "Días", ylab = "Nº de casos")
polygon(c(time_points, rev(time_points)), c(up, rev(low)),
        border = NA, col = adjustcolor("red", alpha.f = 0.1))
lines(time_points,fit_mle$I,col="red")
```



Modelo SIR: estimación por máxima verosimilitud (Poisson)

- En realidad, al tratarse de casos (discretos) podemos usar una distribución de Poisson.
- Para ello tenemos que crear una función nueva que incluya la verosimilitud Poisson.

```
# Definimos una función de la log-verosimilud
mLL_pois <- function(beta, gamma, dias, casos, N = 763) {
   beta <- exp(beta)    # para asegurar parametros >0
  gamma <- exp(gamma)   #  parametros en escala log

  I0 <- casos[1] #
  observations <- casos[-1] # ajustamos los datos restantes
  predictions <- mySIR(beta = beta, gamma = gamma,
                       S0 = N - I0, I0 = I0, R0 = 0, times = dias)
  predictions <- predictions$I[-1] # eliminamos el primer valor ajustado
  if (any(predictions < 0)) return(NA) # En caso de valores negativos =NA
# - log-verosimilitud de una Distribución Poisson:
  -sum(dpois(x = observations, lambda = predictions, log = TRUE))
}
```

Modelo SIR: estimación por máxima verosimilitud (Poisson)

```
# Lista de valores iniciales de los parámetros
starting_param_val <- list(beta = 0.004, gamma = 0.5)

# Estimamos con mle2
estimates_pois <- mle2(minuslogl = mLL_pois,
                       start = lapply(starting_param_val, log),
                       method = "Nelder-Mead", data = c(datos, N = 763))
estimates_pois
##
## Call:
## mle2(minuslogl = mLL_pois, start = lapply(starting_param_val,
##     log), method = "Nelder-Mead", data = c(datos, N = 763))
##
## Coefficients:
##       beta      gamma
##  0.5424941 -0.7418904
##
## Log-likelihood: -72.33
```

```
summary(estimates_pois)
## Maximum likelihood estimation
##
## Call:
## mle2(minuslogl = mLL_pois, start = lapply(starting_param_val,
##     log), method = "Nelder-Mead", data = c(datos, N = 763))
##
## Coefficients:
##        Estimate Std. Error z value     Pr(z)
## beta   0.542494   0.010883  49.849 < 2.2e-16 ***
## gamma -0.741890   0.022962 -32.309 < 2.2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## -2 log L: 144.6607
exp(coef(estimates_pois)) # Distr. Poisson
##      beta     gamma
## 1.7202921 0.4762128
```

```
summary(estimates_pois)
## Maximum likelihood estimation
##
## Call:
## mle2(minuslogl = mLL_pois, start = lapply(starting_param_val,
##     log), method = "Nelder-Mead", data = c(datos, N = 763))
##
## Coefficients:
##        Estimate Std. Error z value     Pr(z)
## beta   0.542494   0.010883  49.849 < 2.2e-16 ***
## gamma -0.741890   0.022962 -32.309 < 2.2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## -2 log L: 144.6607
exp(coef(estimates_pois)) # Distr. Poisson
##      beta     gamma
## 1.7202921 0.4762128
```

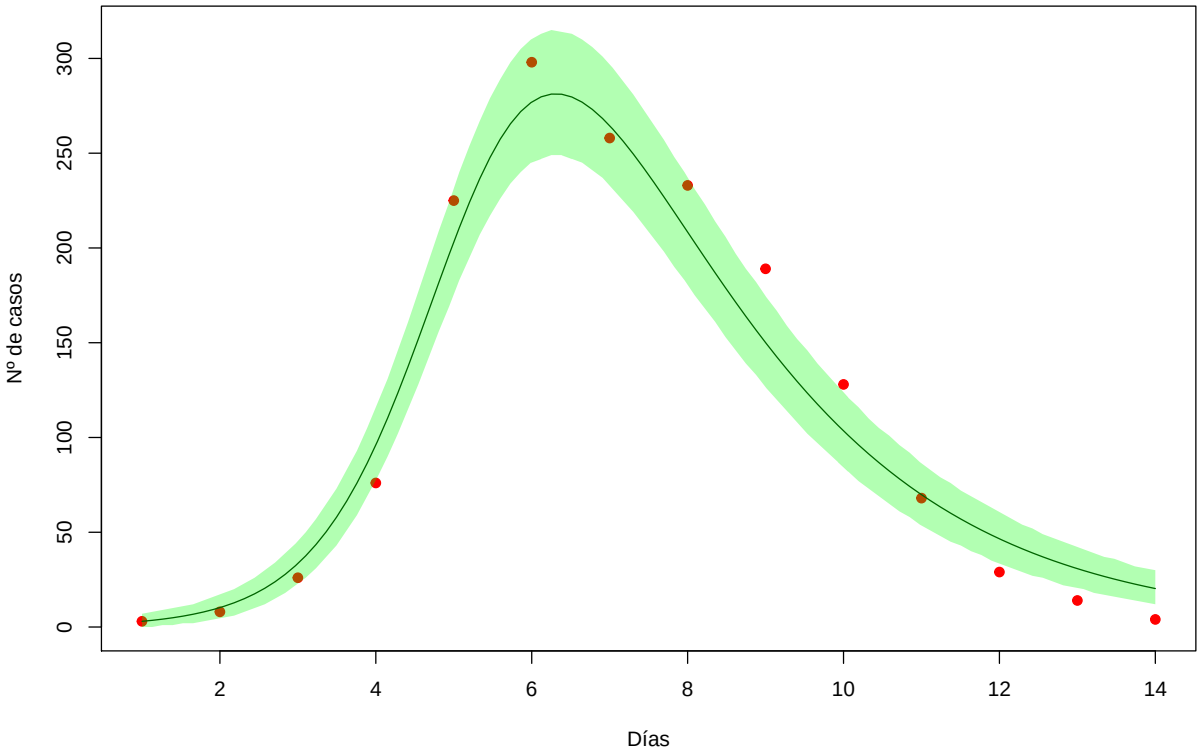
```
exp(coef(estimates))      # Distr. Normal
##       beta      gamma      sigma
##  1.7003425  0.4467678 17.2602308
```

```r
exp(confint(estimates))      # Distr. Normal
##            2.5 %     97.5 %
## beta   1.6413089  1.7606428
## gamma  0.4179379  0.4788757
## sigma 12.2783425 26.8251801
exp(confint(estimates_pois)) # Distr. Poisson
##           2.5 %   97.5 %
## beta  1.6842442 1.757696
## gamma 0.4552564 0.498145
```

```r
 beta_hat_pois <- exp(coef(estimates_pois))[1]
gamma_hat_pois <- exp(coef(estimates_pois))[2]

# Ajustamos el modelo SIR con los parámetros obtenidos por MV
fit_pois <- mySIR(beta = beta_hat_pois,
                  gamma = gamma_hat_pois,
                  S0 = S0, I0 = I0, R0 = 0, times = time_points)
head(fit_pois)
##       time        S        I         R
## 1 1.000000 760.0000 3.000000 0.0000000
## 2 1.131313 759.2674 3.528889 0.2036925
## 3 1.262626 758.4067 4.150041 0.4432668
## 4 1.393939 757.3959 4.879177 0.7249709
## 5 1.525253 756.2093 5.734557 1.0561155
## 6 1.656566 754.8174 6.737330 1.4452406
```

```
low_pois <- qpois(p = 1-0.975,lambda=fit_pois$I)
 up_pois <- qpois(p = 0.975,lambda=fit_pois$I)
```

```
plot(datos$dias,datos$casos,col="red",pch=19, ylim = c(0,max(up_pois)),
     xlab = "Días", ylab = "Nº de casos")
polygon(c(time_points, rev(time_points)), c(up_pois, rev(low_pois)),
        border = NA, col = adjustcolor("green", alpha.f = 0.3))
lines(time_points,fit_pois$I,col="darkgreen")
```



Modelo SIR: estimación por máxima verosimilitud (binomial negativa)

```
mLL_nb <- function(beta, gamma, theta, dias, casos, N = 763) {
  beta <- exp(beta)    # para asegurar parametros >0
gamma <- exp(gamma)   #  parametros en escala log
theta <- exp(theta)
 size <- 1/(theta)    # dispersion parameter
  I0 <- casos[1] #
  observations <- casos[-1] # ajustamos los datos restantes
  predictions <- mySIR(beta = beta, gamma = gamma,
                       S0 = N - I0, I0 = I0, R0 = 0, times = dias)
  predictions <- predictions$I[-1] # eliminamos el primer valor ajustado
  if (any(predictions < 0)) return(NA) # En caso de valores negativos =NA
  mu <- predictions
  prob <- size/(mu+size)
  # - log-verosimilitud de una Distribución Binomial Negativa
  -sum(dnbinom(x = observations, size = size, mu = mu, log = TRUE))
}
```

Modelo SIR: estimación por máxima verosimilitud (binomial negativa)

```
# Lista de valores iniciales de los parámetros
starting_param_val <- list(beta = 4, gamma = 0.5, theta = 1)

# Estimamos con mle2
estimates_nb <- mle2(minuslogl = mLL_nb,
                     start = lapply(starting_param_val, log),
                     method = "Nelder-Mead", data = c(datos, N = 763))
estimates_nb
##
## Call:
## mle2(minuslogl = mLL_nb, start = lapply(starting_param_val, log),
##     method = "Nelder-Mead", data = c(datos, N = 763))
##
## Coefficients:
##       beta      gamma      theta
##  0.5621323 -0.6252640 -2.5414934
##
## Log-likelihood: -58.83
```

```r
summary(estimates_nb)
## Maximum likelihood estimation
##
## Call:
## mle2(minuslogl = mLL_nb, start = lapply(starting_param_val, log),
##     method = "Nelder-Mead", data = c(datos, N = 763))
##
## Coefficients:
##        Estimate Std. Error z value     Pr(z)
## beta   0.562132   0.029412 19.1122 < 2.2e-16 ***
## gamma -0.625264   0.068533 -9.1235 < 2.2e-16 ***
## theta -2.541493   0.569625 -4.4617 8.131e-06 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## -2 log L: 117.67
```

```r
exp(coef(estimates_nb))
##       beta      gamma      theta
## 1.75440952 0.53512015 0.07874871
```

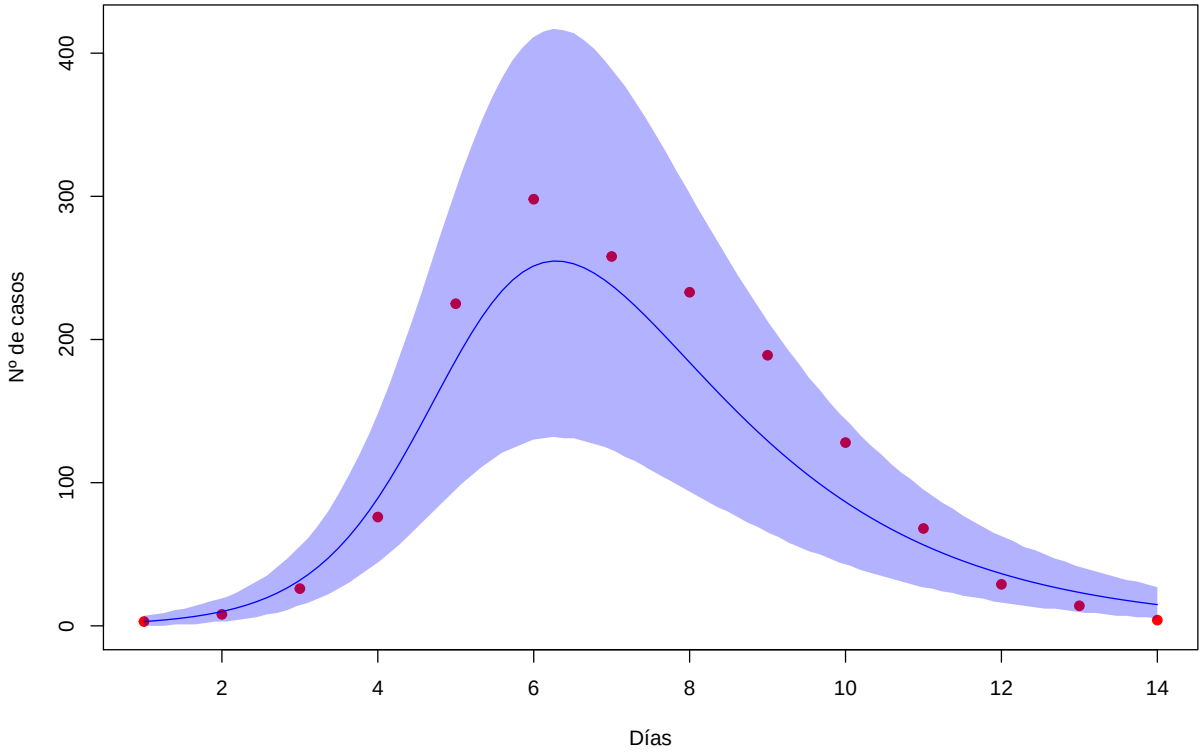
```r
exp(confint(estimates_nb))
##            2.5 %    97.5 %
## beta  1.65218661 1.8806060
## gamma 0.46502146 0.6230156
## theta 0.02482382 0.2439348
```

```r
 beta_hat_nb <- exp(coef(estimates_nb))[1]
gamma_hat_nb <- exp(coef(estimates_nb))[2]
theta_hat_nb <- exp(coef(estimates_nb))[3]

# Parámetro de dispersión
1/theta_hat_nb
##    theta
## 12.69862

# Ajustamos el modelo SIR con los parámetros obtenidos por MV
fit_nb <- mySIR(beta = beta_hat_nb,
                gamma = gamma_hat_nb,
                S0 = S0, I0 = I0, R0 = 0, times = time_points)
```

```r
 beta_hat_nb <- exp(coef(estimates_nb))[1]
gamma_hat_nb <- exp(coef(estimates_nb))[2]
theta_hat_nb <- exp(coef(estimates_nb))[3]

# Parámetro de dispersión
1/theta_hat_nb
##    theta
## 12.69862

# Ajustamos el modelo SIR con los parámetros obtenidos por MV
fit_nb <- mySIR(beta = beta_hat_nb,
                gamma = gamma_hat_nb,
                S0 = S0, I0 = I0, R0 = 0, times = time_points)
```

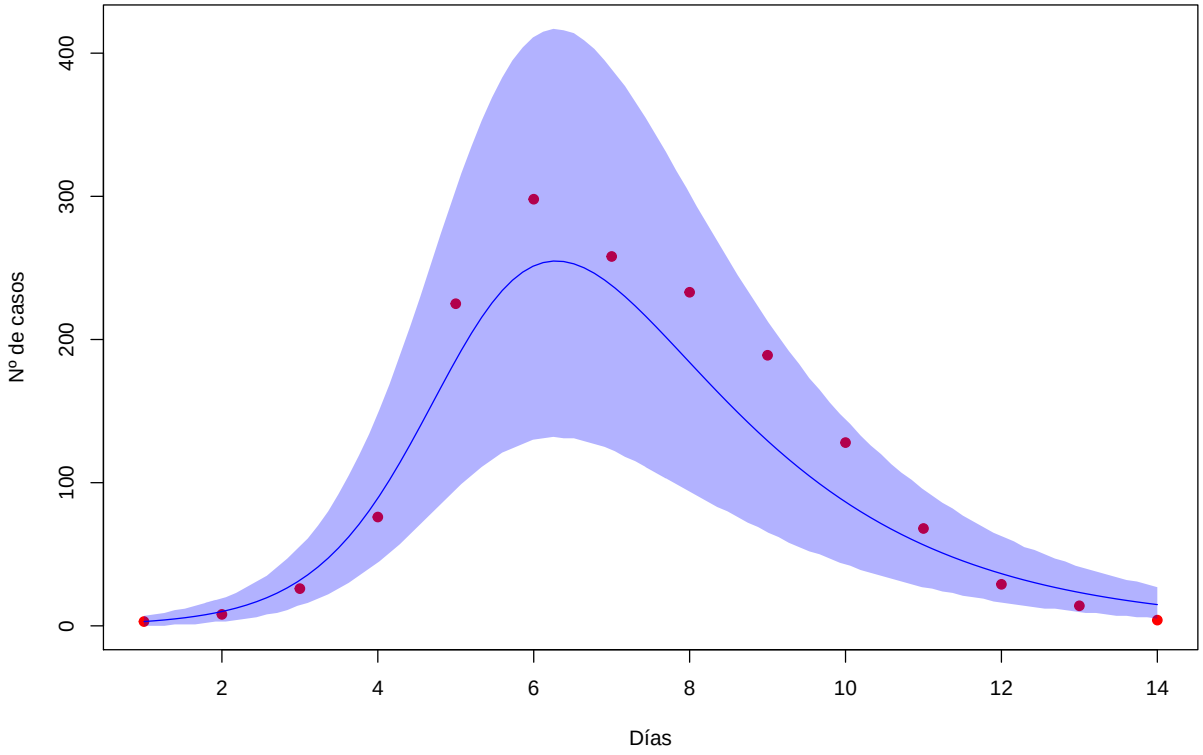
```r
# Calculamos los IC al 95%
 size = 1/theta_hat_nb
 mu = fit_nb$I
 prob = size/(mu+size)

 low_nb <- qnbinom(p = 1-0.975,size = size,prob = prob)
  up_nb <- qnbinom(p = 0.975,size = size,prob = prob)
```

```
plot(datos$dias,datos$casos,col="red",pch=19, ylim = c(0,max(up_nb)),
     xlab = "Días", ylab = "Nº de casos")
polygon(c(time_points, rev(time_points)), c(up_nb, rev(low_nb)),
        border = NA, col = adjustcolor("blue", alpha.f = 0.3))
lines(time_points,(fit_nb$I),col="blue")
```



¿Qué ha ocurrido?

```
plot(datos$dias,datos$casos,col="red",pch=19, ylim = c(0,max(up_nb)),
     xlab = "Días", ylab = "Nº de casos")
polygon(c(time_points, rev(time_points)), c(up_nb, rev(low_nb)),
        border = NA, col = adjustcolor("blue", alpha.f = 0.3))
lines(time_points,(fit_nb$I),col="blue")
```



¿Qué ha ocurrido? Básicamente, el modelo SIR, tal como lo hemos escrito, no puede capturar la forma de la epidemia. Para ajustar los datos, el algoritmo de optimización ha ampliado la varianza del error.

Modelo SIR: estimación por máxima verosimilitud

```
-logLik(estimates)
## 'log Lik.' 55.4782 (df=3)

-logLik(estimates_pois)
## 'log Lik.' 72.33037 (df=2)

-logLik(estimates_nb)
## 'log Lik.' 58.83498 (df=3)

AICtab(estimates,estimates_pois, estimates_nb,logLik = TRUE)
##                dLogLik dAIC df
## estimates      16.9     0.0 3
## estimates_nb   13.5     6.7 3
## estimates_pois  0.0    31.7 2
```

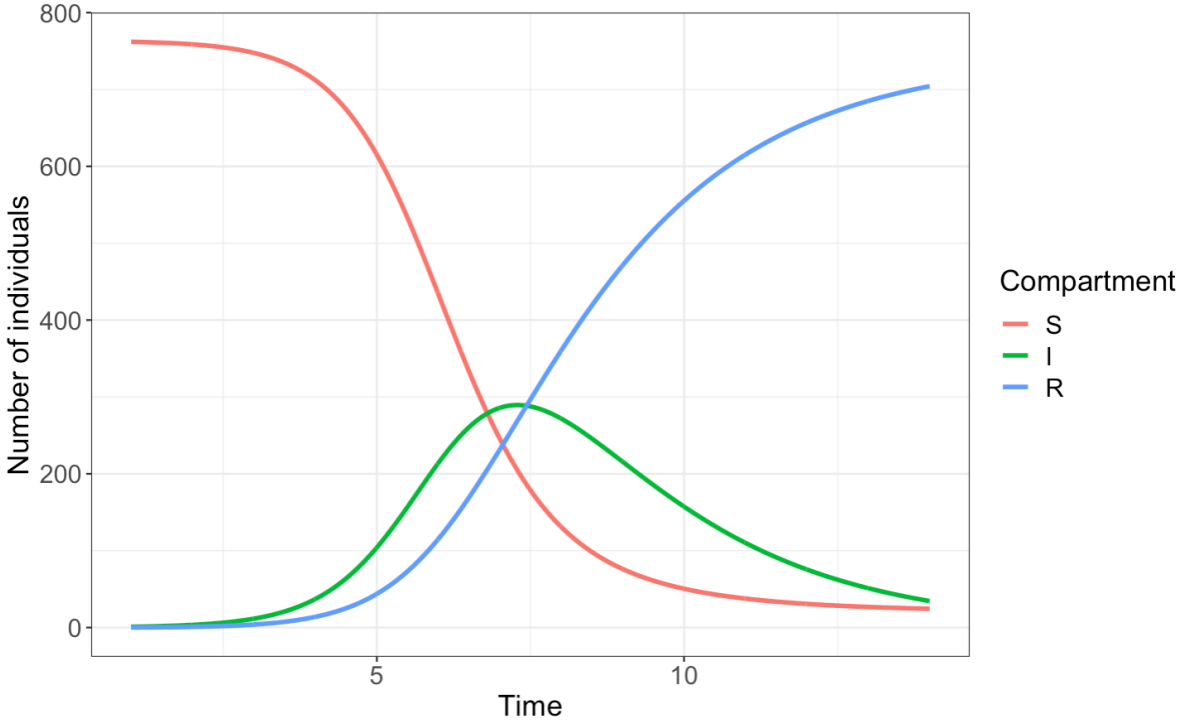
Nº reproductivo básico (R_0)

```
R0 <- beta_hat/gamma_hat
R0
## [1] 3.825503
infectious_period <- 1/gamma_hat
infectious_period
## [1] 2.214765
```

Nº reproductivo básico (R_0)

```
R0 <- beta_hat/gamma_hat
R0
## [1] 3.825503
infectious_period <- 1/gamma_hat
infectious_period
## [1] 2.214765
```

```
# visualización con shinySIR
run_shiny(model = "SIR", ics = c(S=762,I=1,R=0), tstart = 1, tmax=14,
          timestep = 0.01,
          parm0 = c(R0 = R0, Ip = infectious_period),
          parm_min = c(R0 = 0, Ip = 1),parm_max = c(R0 = 8, Ip = 10),
          parm_names = c("R0", "Periodo de Infección"),
          sigfigs = 3, slider_steps = c(0.001,0.001))
```



Parameter	Value
gamma	4.5E-01
beta	2.2E-03

Número reproductivo básico instantáneo (R_t)

- **Número de reproducción básico instantáneo** (R_t), es el número promedio de casos secundarios que cada sujeto infectado puede llegar a infectar en una ventana de tiempo (t).

```
library(EpiEstim)
```

Número reproductivo básico instantáneo (R_t)

- **Número de reproducción básico instantáneo** (R_t), es el número promedio de casos secundarios que cada sujeto infectado puede llegar a infectar en una ventana de tiempo (t).

```
library(EpiEstim)
```

- Enfoque de *Cori et al.* (2013) para estimar R_t a partir de los datos de incidencia y las hipótesis sobre la distribución de los intervalos en serie.
- Enfoque de *Wallinga y Teunis* (2004) para estimar R_t a partir de los datos de incidencia y dada una distribución de intervalos en serie.
- Permite realizar gráficos para visualizar conjuntamente la curva de la epidemia, el número de reproducción estimado y la distribución de intervalos en serie utilizados para la estimación.

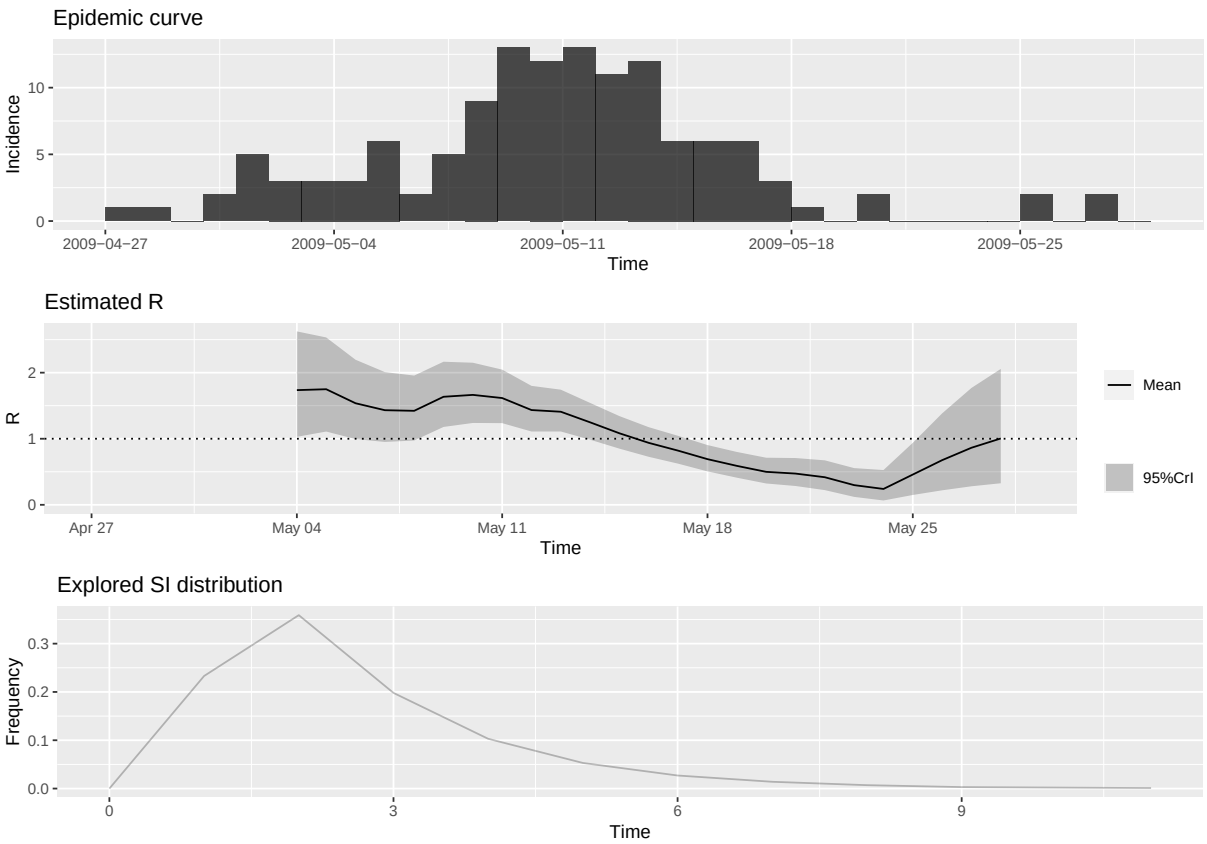
Librería `EpiEstim`

- La función `estimate_R` estima el número reproductivo instantáneo de una epidemia, dada la serie temporal de incidencia y la distribución de intervalos de serie.
- La opción `method` incluye diferentes métodos para estimar la distribución del intervalo serial.
  - **e.g.:** `"non_parametric_si"`, `"parametric_si"`, `"uncertain_si"`, `"si_from_data"` o `"si_from_sample"`.

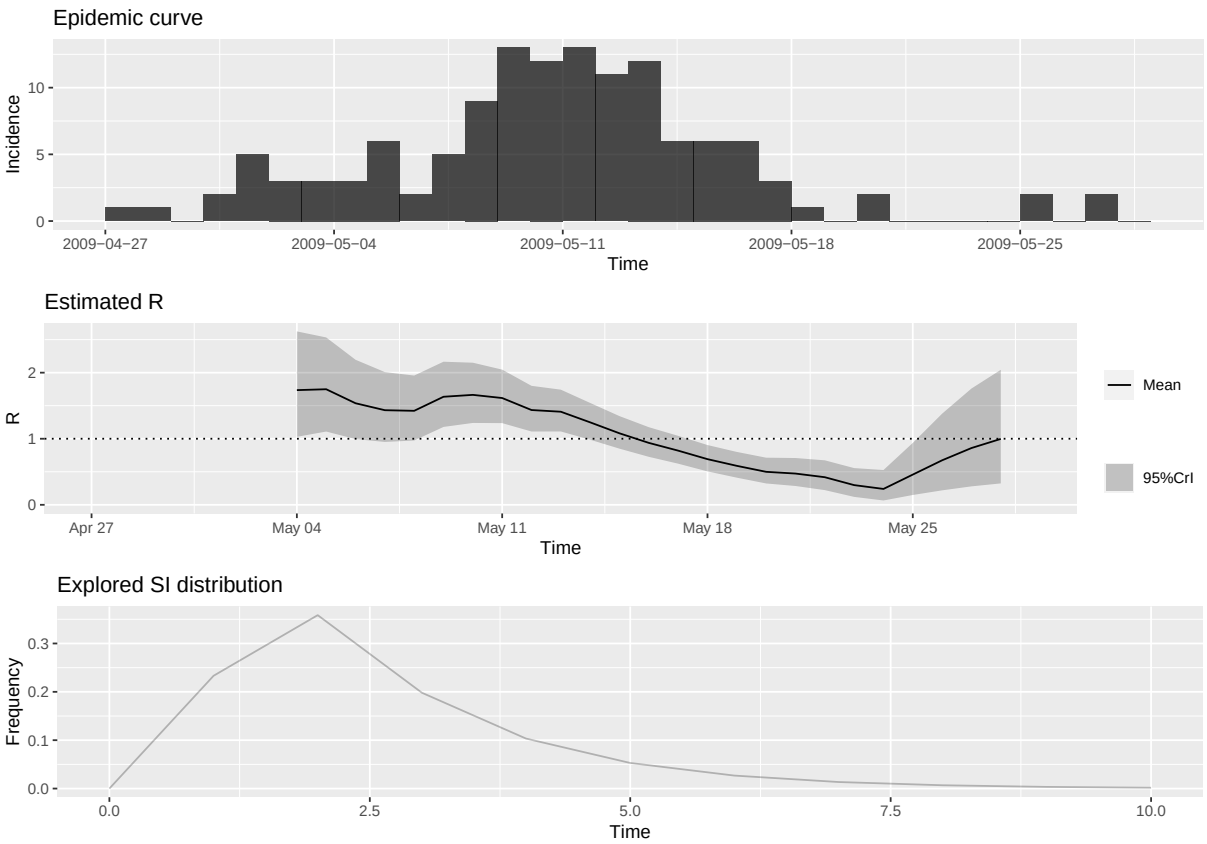
```r
# Data on the 2009 H1N1 influenza pandemic in a school in Pennsylvania.
data("Flu2009")

names(Flu2009)
## [1] "incidence" "si_distr"  "si_data"
```

- `incidence`: incidencia diaria.
- `si_distr`: la distribución diaria discreta del intervalo de serie para la gripe, suponiendo una distribución Gamma desplazada con una media de 2,6 días, una desviación estándar de 1,5 días y un desplazamiento de 1 día.

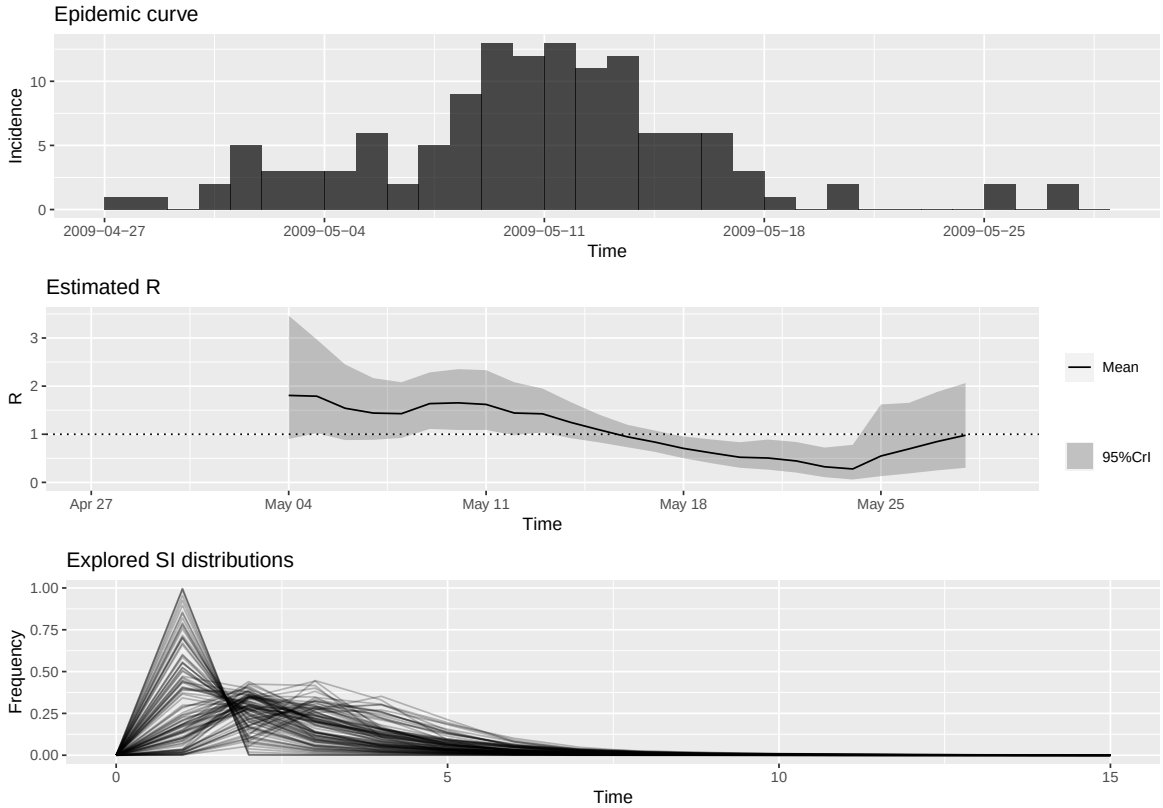
```r
## method "non_parametric_si"
res <- estimate_R(incid = Flu2009$incidence, method = "non_parametric_si",
                  config = make_config(list(si_distr = Flu2009$si_distr)))
plot(res)
```



```r
## method "parametric_si", con supuestos sobre el intervalo serial
res <- estimate_R(Flu2009$incidence, method = "parametric_si",
                  config = make_config(list(mean_si = 2.6, std_si = 1.5)))
plot(res)
```



```r
## estimate the reproduction number (method "uncertain_si")
res <- estimate_R(Flu2009$incidence, method = "uncertain_si",
                  config = make_config(list(
                  mean_si = 2.6, std_mean_si = 1,
                  min_mean_si = 1, max_mean_si = 4.2,
                  std_si = 1.5, std_std_si = 0.5,
                  min_std_si = 0.5, max_std_si = 2.5,
                  n1 = 100, n2 = 100)))
plot(res)
```



Referencias

SIR

- Kermack, W. O., and A. G. McKendrick. 1927. A contribution to the mathematical theory of epidemics. Proceedings of the Royal Society of London, Series A 115:700–721.

Nº Reproductivo

- Cori A., Ferguson, N.M., Fraser, C., Cauchemez, S. *"A New Framework and Software to Estimate Time-Varying Reproduction Numbers During Epidemics"* American Journal of Epidemiologie, Vol 178(9). November 2012, 1505-1512.
- Wallinga J., Teunis, P. *"Different Epidemic Curves for Severe Acute Respiratory Syndrome Reveal Similar Impacts of Control Measures"* American Journal of Epidemiologie, Vol 160(6). September 2004, 509-516.

Estimación

- Lloyd-Smith, J. *"Maximum Likelihood Estimation of the Negative Binomial Dispersion Parameter for Highly Overdispersed Data, with Applications to Infectious Diseases"* Plos One, Feb 2017.